YENEPOYA
(DEEMED TO BE UNIVERSITY)

THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)

AN INTERNSHIP REPORT ON

AI-POWERED RAG CHATBOT FOR COMPANY DATABASE ANALYSIS

SUBMITTED BY

MUHAMMED MIDLAJ P

23BBCAMR303

VI SEMESTER

BCA (AI, ML AND ROBOTICS)

UNDER THE GUIDANCE OF

Dr. R. Kaviyarasi (College Internal Guide)

PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD

OF

THE DEGREE OF BACHELOR IN COMPUTER APPLICATIONS

April 2026

YENEPOYA
(DEEMED TO BE UNIVERSITY)

CERTIFICATE

This is to certify that Mr. Muhammed Midlaj P, bearing Registration Number 23BBCAMR303, of Batch 2023–2026, is a bonafide student of the Bachelor of Computer Applications (BCA) programme at Yenepoya Institute of Arts, Science, Commerce and Management, affiliated to Yenepoya (Deemed to be University).

He has successfully completed the internship titled "[Title of the Report]" at Scanntek IT Solutions LLP, Kinfra Techno Industrial Park, Calicut under the guidance of Dr. R. Kaviyarasi.

This internship report is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications at Yenepoya (Deemed to be University), Bengaluru, Karnataka.

Signature of Internal Guide **Signature of HOD**

Signature of Vice Principal **Signature of Director-Academics**

YENEPOYA
(DEEMED TO BE UNIVERSITY)

DECLARATION

I, Mr. Muhammed Midlaj P, bearing Registration Number 23BBCAMR303, hereby declare that the internship report titled "[Title of the Project]", with reference to Scanntek IT Solutions LLP, has been prepared by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) at Yenepoya (Deemed to be University).

I further declare that this internship report is the result of my own work carried out under the guidance of Dr. R. Kaviyarasi, Professor, Yenepoya Institute of Arts, Science, Commerce and Management.

I also confirm that this report has not been previously submitted to any university or institution for the award of any degree or diploma.

Place: Bengaluru

Date: 16-04-2026

Name : Muhammed Midlaj P

Reg No : 23BBCAMR303

Campus ID : 31058

YENEPOYA
(DEEMED TO BE UNIVERSITY)

ACKNOWLEDGEMENT

Firstly, I Would like to express my immense gratitude towards my institution **Yenepoya Deemed to be University** which created a great platform to attain profound technical skills in the fields of Computer Applications, there by fulfilling my most cherished goal.

I am thankful to **Mr. Moideen Khurshid Yenepoya** (Managing Director) YDU, Bengaluru Campus and Management for providing the necessary infrastructure and labs.

I feel privileged to offer my sincere thanks and deep sense of gratitude to **Dr. Jerry John** (Director-Academics), **Dr. Melita L** (Vice Principal, YIASCM, Bengaluru Campus), **Mr. Syed Umar Abbasi** (HOD, Department of Computer Science & Information Technology, Bengaluru Campus), **Dr. R Kaviyarasi** (Academic Head, Department of Computer Science & Information Technology, Bengaluru Campus) for extending their co-operation in doing this project.

I express my sincere gratitude and thankfulness towards my project guide, **Dr. R. Kaviyarasi, Professor,** for her valuable time and guidance throughout the BCA project.

My special thanks to all the faculty members of the Department of Computer Science & Information Technology for their valuable advises at every stage.

I would like to express my deep sense and earnest thanks giving to my dear Parents and my family members for their moral support and kind cooperation in doing the project.

I would also like to thank all my friends, whose direct or indirect help has enabled me to complete this project.

Lastly, I would like to express my deepest gratitude to my family and friends for their unwavering support, encouragement, and understanding throughout my internship journey. Their belief in my abilities and constant motivation have been the driving force behind my success Thank you all once again for your invaluable support.

Place: Bengaluru

Name : Muhammed Midlaj P

Campus ID: 31058

Date: 16-04-2026

Reg No : 23BBCAMR303

TABLE OF CONTENTS

INTRODUCTION **1**

1.1 Introduction 2

1.2 Previous System 2

1.3 Proposed System 2

1.4 Description/Modules (if any) 2

2. LITERATURE SURVEY 3

2.1 Introduction of what is your system 4

2.2 About Existing Model 4

2.3 Software/ Tools overview 4

2.3.1 Software 4

3. DESIGN 5

3.1 Unified Modelling Language 6

3.1.1 Class Design 6

3.1.2 Use Case 6

4. IMPLEMENTATION 7

4.1 Implementation 8

4.2 Sample Code 8

4.3 Screen Shots 8

ABSTRACT

The rapid proliferation of digital documents within modern enterprises has created an urgent need for intelligent knowledge retrieval systems that can bridge the gap between vast unstructured data repositories and the information needs of end-users. Traditional keyword-based search mechanisms often fail to capture the semantic nuances of user queries, leading to irrelevant results and wasted productivity.

This internship report presents the design, development, and deployment of an AI-Powered Retrieval-Augmented Generation (RAG) Chatbot for Company Database Analysis. The system leverages cutting-edge artificial intelligence technologies, including Google Gemini 2.5 Flash as the core Large Language Model (LLM), Gemini Embedding 2.0 for generating high-dimensional semantic embeddings (3072-dimensional vectors), and Qdrant as a distributed, scalable vector database for efficient similarity search.

The backend is architected using Python 3.11 with Django 5 and Django REST Framework, providing a robust API layer. The frontend is built with React 19, TypeScript 5, and Vite 7, delivering a responsive and modern single-page application. The entire system is containerized using Docker and Docker Compose, ensuring consistent deployment across development and production environments.

Key features of the system include multi-format document ingestion (PDF, DOCX, CSV, XLSX, TXT, Markdown), agentic semantic search with a ReAct (Reasoning and Acting) loop for intelligent query processing, real-time response streaming via Server-Sent Events (SSE), multimodal RAG capabilities supporting image-based queries, role-based access control (Admin, Editor, Viewer), comprehensive analytics dashboard, and JWT-based authentication with rate limiting.

The project follows a three-phase development methodology spanning twelve weeks, from MVP development through enhanced RAG capabilities to production-grade scaling and optimization. Testing strategies including unit testing, integration testing, and system testing were employed to ensure reliability. The system successfully achieves its objective of democratizing access to company knowledge through an intelligent, conversational AI interface that understands context and provides trustworthy, source-cited answers.

TABLE OF CONTENTS

Abstract i

Table of Contents ii

List of Figures iii

List of Tables iv

List of Abbreviations v

1. Introduction 1

1.1 Background of the Problem 1

1.2 Need for the System 3

1.3 Objectives of the Project 4

1.4 Scope and Limitations 5

2. Literature Survey 7

2.1 Secondary Research 7

2.2 Existing Systems and Limitations 9

2.3 Comparative Analysis 11

3. Specifications Overview 13

3.1 Hardware Requirements 13

3.2 Software Requirements 14

3.3 Tools and Technologies Used 15

4. Requirement Analysis 18

4.1 Functional Requirements 18

4.2 Non-Functional Requirements 22

5. System Design 25

5.1 System Architecture 25

5.2 Flowcharts 27

5.3 Data Flow Diagrams (DFD) 29

5.4 Database Design (ERD) 31

5.5 UML Design 33

5.6 Use Case Diagrams 35

5.7 Sequence Diagrams 37

5.8 Class Diagrams 39

5.9 Activity Diagrams 41

6. Implementation 43

6.1 Backend Implementation 43

6.2 Frontend Implementation 50

6.3 RAG Pipeline Implementation 53

6.4 Output Screenshots 57

7. Test Cases 61

7.1 Unit Testing 61

7.2 Integration Testing 63

7.3 System Testing 65

8. Feature Enhancement 67

9. Conclusion 69

10. Bibliography 71

LIST OF FIGURES

Figure 5.1: System Architecture Diagram

Figure 5.2: Application Flowchart

Figure 5.3: Data Flow Diagram (DFD) – Level 1

Figure 5.4: Database Design – Entity Relationship Diagram

Figure 5.5: UML Design Overview

Figure 5.6: Use Case Diagram

Figure 5.7: Sequence Diagram

Figure 5.8: Class Diagram

Figure 5.9: Activity Diagram

Figure 6.1: Landing Page

Figure 6.2: Landing Page – Features Section

Figure 6.3: Landing Page – Call to Action

Figure 6.4: User Registration Page

Figure 6.5: User Login Page

Figure 6.6: Chat Interface

Figure 6.7: Document Upload & Management

Figure 6.8: Admin Dashboard

Figure 6.9: User Management Panel

Figure 6.10: User Profile Section

LIST OF TABLES

Table 3.1: Hardware Requirements

Table 3.2: Software Requirements

Table 3.3: Backend Technology Stack

Table 3.4: Frontend Technology Stack

Table 3.5: Infrastructure & DevOps Stack

Table 4.1: Functional Requirements Summary

Table 4.2: Non-Functional Requirements

Table 4.3: User Roles and Permissions

Table 7.1: Unit Test Cases

Table 7.2: Integration Test Cases

Table 7.3: System Test Cases

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
RAG	RETRIEVAL-AUGMENTED GENERATION
LLM	LARGE LANGUAGE MODEL
AI	ARTIFICIAL INTELLIGENCE
ML	MACHINE LEARNING
NLP	NATURAL LANGUAGE PROCESSING
API	APPLICATION PROGRAMMING INTERFACE
REST	REPRESENTATIONAL STATE TRANSFER
JWT	JSON WEB TOKEN
SSE	SERVER-SENT EVENTS
DFD	DATA FLOW DIAGRAM
UML	UNIFIED MODELING LANGUAGE
ERD	ENTITY RELATIONSHIP DIAGRAM
CRUD	CREATE, READ, UPDATE, DELETE
URL	UNIFORM RESOURCE LOCATOR
HTML	HYPERTEXT MARKUP LANGUAGE
CSS	CASCADING STYLE SHEETS
SQL	STRUCTURED QUERY LANGUAGE
ORM	OBJECT-RELATIONAL MAPPING
SPA	SINGLE PAGE APPLICATION
TTFT	TIME TO FIRST TOKEN
DBMS	DATABASE MANAGEMENT SYSTEM
CPU	CENTRAL PROCESSING UNIT
GPU	GRAPHICS PROCESSING UNIT
RAM	RANDOM ACCESS MEMORY
SDK	SOFTWARE DEVELOPMENT KIT

CI/CD	CONTINUOUS INTEGRATION / CONTINUOUS DEPLOYMENT
CORS	CROSS-ORIGIN RESOURCE SHARING
CSRF	CROSS-SITE REQUEST FORGERY
XSS	CROSS-SITE SCRIPTING
HSTS	HTTP STRICT TRANSPORT SECURITY
CSP	CONTENT SECURITY POLICY

CHAPTER 1: INTRODUCTION

1.1 Background of the Problem

1.1 Background of the Problem

In today's fast-paced corporate environment, organizations generate and accumulate vast quantities of digital documents, including reports, policy manuals, technical specifications, financial statements, and internal communications. According to a study by McKinsey Global Institute, knowledge workers spend approximately 19% of their time searching for and gathering information, which translates to nearly one full day of every work week lost to inefficient information retrieval.

Traditional search mechanisms employed in enterprise settings, such as keyword-based file searches and manual document browsing, suffer from several critical limitations. These systems rely on exact string matching, which entirely misses semantically related content. For example, a search for 'employee leave policy' would not return a document titled 'Staff Vacation Guidelines' despite covering the same topic. This semantic gap between user intent and retrieval capability represents a fundamental barrier to knowledge accessibility.

The emergence of Large Language Models (LLMs) such as Google Gemini, OpenAI GPT, and Meta LLaMA has opened new frontiers in natural language understanding and generation. However, standalone LLMs face a critical limitation known as 'hallucination,' where the model generates plausible but factually incorrect information. This is particularly dangerous in enterprise contexts where accuracy is paramount, as decisions based on fabricated data could lead to financial losses, compliance violations, or operational failures.

Retrieval-Augmented Generation (RAG) addresses this hallucination problem by grounding LLM responses in actual retrieved documents. By combining the retrieval capabilities of vector databases with the generative prowess of LLMs, RAG systems provide accurate, source-cited, and contextually relevant answers. This architectural paradigm ensures that every response can be traced back to its source document, providing the transparency and trustworthiness required in professional settings.

The concept of RAG was formally introduced by Lewis et al. (2020) in their seminal paper 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' which demonstrated that augmenting language models with a non-parametric retrieval component significantly improved factual accuracy. Since then, the RAG paradigm has evolved rapidly, incorporating advances in dense retrieval, multi-step reasoning, and agentic architectures that allow the system to iteratively refine its search strategy.

1.2 Need for the System

1.2 Need for the System

The need for an AI-Powered RAG Chatbot arises from several pressing challenges faced by modern organizations:

• Information overload: organizations accumulate terabytes of documents across various formats (PDF, DOCX, CSV, XLSX, TXT, markdown). Manually searching through these repositories is time-consuming and error-prone, often resulting in critical information being overlooked.

• Semantic search gap: existing enterprise search tools rely primarily on keyword matching, which fails to understand the semantic meaning behind user queries. A user asking 'what is our policy on remote work?' Expects results about telecommuting guidelines, flexible scheduling, and work-from-home procedures, regardless of the specific terminology used in the source documents.

• Knowledge silos: information is often trapped in departmental silos, with different teams using different storage systems and naming conventions. This fragmentation makes cross-departmental knowledge sharing virtually impossible without A unified search interface.

• Lack of contextual understanding: traditional systems cannot maintain conversation context or understand follow-up questions. Users must reformulate their queries from scratch for each interaction, losing the natural flow of inquiry that characterizes human conversation.

• Absence of source verification: when employees share information obtained from internal documents, there is often no easy way to verify the source or check for updates. This leads to the propagation of outdated or incorrect information across the organization.

• Scalability concerns: as document volumes grow, the performance of traditional search mechanisms degrades significantly. A system built on modern vector database technology (such as qdrant) can maintain sub-second search latency even with millions of document chunks.

1.3 Objectives of the Project

1.3 Objectives of the Project

The primary objectives of this project are as follows:

1. To design and develop an enterprise-grade RAG chatbot that enables natural language querying of company documents with AI-powered, contextually accurate responses.

2. To implement a multi-format document ingestion pipeline supporting PDF, DOCX, CSV, XLSX, TXT, and Markdown files with automatic text extraction, intelligent chunking, and semantic embedding using Google Gemini Embedding 2.0.

3. To build an agentic semantic search system utilizing a ReAct (Reasoning and Acting) loop architecture that intelligently rewrites queries, performs multi-step retrieval, and self-evaluates observations before generating final answers.

4. To develop a responsive, modern web frontend using React 19 with TypeScript that provides real-time streamed responses via Server-Sent Events (SSE), conversation history management, and an intuitive document management interface.

5. To implement comprehensive security measures including JWT-based authentication, role-based access control (Admin, Editor, Viewer), rate limiting, CSRF protection, XSS sanitization, and Content Security Policy headers.

6. To containerize the entire application stack using Docker and Docker Compose for consistent, reproducible deployments across development and production environments.

7. To provide an analytics dashboard for administrators to monitor system usage, query patterns, document statistics, and user activity metrics.

8. To ensure source transparency by providing inline citations and confidence scores with every AI-generated response, enabling users to verify information against original documents.

1.4 Scope and Limitations

1.4 Scope and Limitations

Scope:

The system is designed to function as a self-hosted, enterprise-grade knowledge management platform. It supports multi-format document upload and processing, real-time conversational AI with source citations, multimodal capabilities (text and image inputs), role-based user access control, comprehensive query history and analytics, and containerized deployment via Docker. The system can be deployed on-premise or on cloud infrastructure, making it suitable for organizations with varying IT infrastructure and data privacy requirements.

Limitations:

- The system currently supports english language queries only; multi-language support is planned for future releases.
- Document processing is limited to text-based content extraction; complex layouts with embedded charts or handwritten text may not be fully parsed.
- The system depends on google gemini API for LLM inference, which introduces external dependency and potential rate limiting constraints on the free tier (1,500 requests/day).
- Real-time collaboration features (multiple users editing the same document simultaneously) are not within the current scope.
- The system does not currently support voice input or output capabilities, though this is identified as a future enhancement.
- Offline operation is not supported, as the system requires internet connectivity for LLM API calls.

CHAPTER 2: LITERATURE SURVEY

2.1 Secondary Research

2.1 Secondary Research

A comprehensive literature survey was conducted to understand the theoretical foundations, existing implementations, and state-of-the-art advancements in the fields of Natural Language Processing (NLP), Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and vector database technologies. The following sections summarize the key findings from this research.

2.1.1 Large Language Models (LLMs)

The foundation of modern conversational AI lies in the Transformer architecture, introduced by Vaswani et al. (2017) in the landmark paper 'Attention Is All You Need.' This architecture revolutionized NLP by replacing recurrent neural networks with self-attention mechanisms, enabling parallel processing of input sequences and capturing long-range dependencies more effectively. The Transformer architecture forms the basis for all contemporary LLMs, including Google's Gemini family, OpenAI's GPT series, and Meta's LLaMA models.

Google Gemini, the LLM used in this project, represents a significant advancement in multimodal AI. The Gemini 2.5 Flash model offers a context window of up to 1 million tokens, native multimodal understanding (text, image, audio, video), and advanced reasoning capabilities. Its efficiency-optimized architecture makes it particularly suitable for enterprise applications where cost and latency are critical considerations.

Brown et al. (2020) demonstrated through GPT-3 that scaling language models to billions of parameters enables few-shot and zero-shot learning capabilities, where the model can perform tasks it was never explicitly trained on. This finding has profound implications for enterprise applications, as it allows a single model to handle diverse query types without task-specific fine-tuning.

2.1.2 Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) introduced the RAG paradigm, which combines the parametric knowledge stored in LLM weights with non-parametric knowledge retrieved from external document collections. This hybrid approach addresses the hallucination problem inherent in standalone LLMs by grounding responses in actual retrieved evidence. The original RAG architecture consists of two components: a retriever (typically based on Dense Passage Retrieval) and a generator (a sequence-to-sequence model).

Gao et al. (2023) published a comprehensive survey on RAG, categorizing the evolution into three paradigms: Naive RAG, Advanced RAG, and Modular RAG. Naive RAG follows a simple retrieve-then-generate pipeline. Advanced RAG introduces pre-retrieval optimizations (query rewriting, expansion) and post-retrieval refinements (re-ranking, compression). Modular RAG, which our system implements, allows flexible orchestration of retrieval, generation, and reasoning modules through agentic architectures.

The ReAct framework, proposed by Yao et al. (2022), combines reasoning and acting in language model agents. This approach enables the model to generate verbal reasoning traces that inform its search actions, observe the results, and iteratively refine its approach. Our system implements a ReAct-based agent loop with a maximum of three iterations, where the agent can autonomously decide to search, evaluate results, and formulate grounded answers.

2.1.3 Vector Databases and Semantic Search

Vector databases are purpose-built storage systems optimized for high-dimensional vector operations, particularly approximate nearest neighbor (ANN) search. Unlike traditional relational databases that operate on structured data with SQL queries, vector databases enable semantic similarity search by comparing the mathematical distance between dense vector representations of text.

Qdrant, the vector database used in this project, is an open-source, distributed vector similarity search engine built in Rust for performance and reliability. It supports filtered search with payload indices, enabling namespace-based isolation where each user's documents are segregated through metadata filtering. Qdrant uses the HNSW (Hierarchical Navigable Small World) algorithm for approximate nearest neighbor search, achieving sub-millisecond query latency even with millions of vectors.

Johnson et al. (2019) introduced FAISS (Facebook AI Similarity Search), which established foundational algorithms for efficient similarity search at scale. Subsequent systems like Pinecone, Weaviate, Milvus, and Qdrant have built upon these foundations to offer distributed, production-ready vector database solutions with rich API ecosystems.

2.1.4 Web Application Frameworks

Django, the backend framework chosen for this project, is a high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern. Created by Adrian Holovaty and Simon Willison in 2005, Django emphasizes rapid development, clean design, and the principle of 'Don't Repeat Yourself' (DRY). Django REST Framework (DRF), built on top of Django, provides a powerful toolkit for building Web APIs with features such as serialization, authentication, viewsets, and routing.

React, developed by Meta (formerly Facebook) in 2013, is a component-based JavaScript library for building user interfaces. React 19, used in this project, introduces several performance improvements including automatic batching, concurrent rendering, and improved server-client integration. Combined with TypeScript for type safety and Vite for efficient development builds, React provides a robust foundation for building interactive, responsive single-page applications.

2.2 Existing Systems and Limitations

2.2 Existing Systems and Limitations

Several existing systems attempt to address the challenge of enterprise knowledge retrieval. However, each has notable limitations that our system aims to overcome:

2.2.1 ChatGPT and General-Purpose LLMs

While ChatGPT and similar general-purpose LLMs excel at generating human-like text, they have three critical limitations for enterprise use: (1) they cannot access private company documents, (2) they are prone to hallucination when answering domain-specific questions, and (3) they do not provide source citations, making it impossible to verify the accuracy of responses. Our system addresses all three limitations through the RAG architecture.

2.2.2 Traditional Enterprise Search (SharePoint, Confluence)

Enterprise search platforms like Microsoft SharePoint and Atlassian Confluence provide keyword-based search capabilities within their respective ecosystems. However, these systems lack semantic understanding, cannot provide synthesized answers to complex questions, and require users to manually review multiple documents to find answers. Our system provides direct, contextual answers with source citations, eliminating the need for manual document review.

2.2.3 Existing RAG Solutions (Langflow, Flowise)

No-code RAG platforms like Langflow and Flowise offer visual pipeline builders for creating RAG applications. While they lower the barrier to entry, they lack the customization depth required for enterprise-grade deployments, including advanced security features, role-based access control, multimodal support, and agentic reasoning capabilities. Our system provides a fully custom, production-grade implementation tailored to specific enterprise requirements.

2.3 Comparative Analysis

2.3 Comparative Analysis

FEATURE	OUR SYSTEM	CHATGPT	SHAREPOINT SEARCH	LANGFLOW
PRIVATE DOCUMENT ACCESS	YES	NO	YES	YES
SOURCE CITATIONS	YES	NO	NO	PARTIAL
SEMANTIC SEARCH	YES (3072-DIM)	N/A	NO	YES

AGENTIC REASONING	YES (REACT)	PARTIAL	NO	NO
MULTIMODAL SUPPORT	YES	YES	NO	NO
ROLE-BASED ACCESS	YES	NO	YES	NO
REAL-TIME STREAMING	YES (SSE)	YES	NO	PARTIAL
SELF-HOSTED	YES	NO	PARTIAL	YES
ANALYTICS DASHBOARD	YES	NO	PARTIAL	NO
CUSTOM SECURITY	JWT + RBAC	N/A	AZURE AD	BASIC

Table 2.1: Comparative Analysis of Knowledge Retrieval Systems

CHAPTER 3: SPECIFICATIONS OVERVIEW

This chapter provides a comprehensive overview of the hardware and software specifications required for the development, deployment, and operation of the AI-Powered RAG Chatbot system.

3.1 Hardware Requirements

3.1 Hardware Requirements

COMPONENT	MINIMUM REQUIREMENT	RECOMMENDED
PROCESSOR	INTEL CORE I5 / AMD RYZEN 5 (4 CORES)	INTEL CORE I7 / AMD RYZEN 7 (8 CORES)
RAM	8 GB DDR4	16 GB DDR4 OR HIGHER
STORAGE	256 GB SSD	512 GB SSD OR HIGHER
NETWORK	BROADBAND INTERNET (10 MBPS)	HIGH-SPEED INTERNET (50+ MBPS)
GPU	NOT REQUIRED (API-BASED INFERENCE)	NVIDIA GPU (OPTIONAL, FOR LOCAL MODELS)
DISPLAY	1366 X 768 RESOLUTION	1920 X 1080 FULL HD

Table 3.1: Hardware Requirements

For production deployment, the following server-side hardware is recommended:

RESOURCE	MINIMUM	RECOMMENDED
BACKEND SERVER	2 VCPU, 4 GB RAM	4 VCPU, 8 GB RAM
POSTGRESQL DATABASE	1 VCPU, 2 GB RAM, 50 GB SSD	2 VCPU, 4 GB RAM, 100 GB SSD
REDIS CACHE	512 MB RAM	1 GB RAM
QDRANT VECTOR DB	2 GB RAM, 20 GB SSD	4 GB RAM, 100 GB SSD
STORAGE VOLUME	100 GB	500 GB (SCALES WITH DOCUMENTS)

3.2 Software Requirements

3.2 Software Requirements

SOFTWARE	VERSION	PURPOSE

OPERATING SYSTEM	UBUNTU 22.04 LTS / WINDOWS 10+	DEVELOPMENT AND SERVER
PYTHON	3.11+	BACKEND PROGRAMMING LANGUAGE
NODE.JS	20 LTS	FRONTEND BUILD TOOLING
DOCKER	24.0+	CONTAINERIZATION PLATFORM
DOCKER COMPOSE	2.20+	MULTI-CONTAINER ORCHESTRATION
POSTGRESQL	15+	RELATIONAL DATABASE
REDIS	7.0+	CACHING AND TASK QUEUE
QDRANT	LATEST	VECTOR DATABASE
GIT	2.40+	VERSION CONTROL
VS CODE / IDE	LATEST	DEVELOPMENT ENVIRONMENT

Table 3.2: Software Requirements

3.3 Tools and Technologies Used

3.3 Tools and Technologies Used

3.3.1 Backend Technology Stack

CATEGORY	TECHNOLOGY	VERSION	JUSTIFICATION
LANGUAGE	PYTHON	3.11+	LANGCHAIN COMPATIBILITY, RICH AI/ML ECOSYSTEM
FRAMEWORK	DJANGO	5.0+	MATURE, BATTERIES-INCLUDED, POWERFUL ORM
API LAYER	DJANGO REST FRAMEWORK	3.14+	INDUSTRY-STANDARD REST API TOOLKIT

RAG FRAMEWORK	LANGCHAIN	0.3+	COMPREHENSIVE RAG PIPELINE ORCHESTRATION
LLM CLIENT	LANGCHAIN-GOOGLE-GENAI	2.0+	NATIVE GEMINI INTEGRATION
VECTOR DB CLIENT	LANGCHAIN-QDRANT	0.1+	QDRANT INTEGRATION FOR LANGCHAIN
TASK QUEUE	DJANGO-Q2	1.6+	ASYNC DOCUMENT PROCESSING
DATABASE DRIVER	PSYCOPG2-BINARY	2.9+	POSTGRESQL ADAPTER FOR PYTHON
CACHE	DJANGO-REDIS	5.4+	REDIS INTEGRATION FOR DJANGO
AUTH	DJANGORESTFRAMEWORK-SIMPLEJWT	5.3+	JWT-BASED STATELESS AUTHENTICATION
API DOCS	DRF-SPECTACULAR	0.27+	OPENAPI 3.0 SCHEMA GENERATION
LOGGING	STRUCTLOG	24.0+	STRUCTURED JSON LOGGING
MONITORING	SENTRY-SDK	2.0+	ERROR TRACKING AND PERFORMANCE

Table 3.3: Backend Technology Stack

3.3.2 Frontend Technology Stack

CATEGORY	TECHNOLOGY	VERSION	JUSTIFICATION
FRAMEWORK	REACT	19.0	LATEST FEATURES, CONCURRENT RENDERING
LANGUAGE	TYPESCRIPT	5.5+	STATIC TYPE SAFETY, IMPROVED DX

BUILD TOOL	VITE	7.0+	ULTRA-FAST HMR, OPTIMIZED BUILDS
STYLING	TAILWIND CSS	4.0+	UTILITY-FIRST, RAPID DEVELOPMENT
UI COMPONENTS	SHADCN/UI + RADIX	LATEST	ACCESSIBLE, CUSTOMIZABLE PRIMITIVES
STATE MANAGEMENT	ZUSTAND	5.0+	LIGHTWEIGHT, NO BOILERPLATE
SERVER STATE	TANSTACK QUERY	5.0+	DATA FETCHING, CACHING, SYNC
ROUTING	REACT ROUTER	7.0+	FILE-BASED ROUTING, LAZY LOADING
FORMS	REACT HOOK FORM + ZOD	LATEST	PERFORMANT VALIDATION
HTTP CLIENT	AXIOS	1.6+	REQUEST INTERCEPTORS, ERROR HANDLING
ANIMATIONS	FRAMER MOTION	12.0+	SMOOTH MICRO-ANIMATIONS
3D GRAPHICS	SPLINE RUNTIME	1.12+	INTERACTIVE 3D HERO SCENES
ICONS	LUCIDE REACT	LATEST	MODERN, CONSISTENT ICONOGRAPHY
MARKDOWN	REACT MARKDOWN	10.0+	AI RESPONSE FORMATTING

Table 3.4: Frontend Technology Stack

3.3.3 Infrastructure & DevOps

CATEGORY	TECHNOLOGY	PURPOSE
CONTAINERIZATION	DOCKER + DOCKER COMPOSE	APPLICATION PACKAGING AND ORCHESTRATION

CI/CD	GITHUB ACTIONS	AUTOMATED TESTING AND DEPLOYMENT PIPELINES
VERSION CONTROL	GIT + GITHUB	SOURCE CODE MANAGEMENT AND COLLABORATION
CODE QUALITY	RUFF, ESLINT, PRETTIER	LINTING AND CODE FORMATTING
TESTING	PYTEST (BACKEND), VITEST (FRONTEND)	UNIT AND INTEGRATION TESTING
API DOCUMENTATION	SWAGGER UI (DRF-SPECTACULAR)	INTERACTIVE API DOCUMENTATION

Table 3.5: Infrastructure & DevOps Stack

CHAPTER 4: REQUIREMENT ANALYSIS

This chapter provides a detailed analysis of the functional and non-functional requirements of the AI-Powered RAG Chatbot system. The requirements have been gathered through secondary research, analysis of existing systems, and consultation with the project guide and industry mentors.

4.1 Functional Requirements

4.1 Functional Requirements

4.1.1 Natural Language Query Processing (FR-001)

The system shall accept free-text input queries from users in plain English, with a maximum length of 2,000 characters. The system must support follow-up questions with context retention, handle spelling errors and informal language gracefully, and reject harmful or inappropriate queries. The query processing module implements intelligent query rewriting, keyword extraction, and intent classification (factual, summary, comparison, list, how-to, general) to optimize retrieval accuracy.

4.1.2 Context-Aware Responses (FR-002)

The system shall maintain conversation history within a session, retaining context for up to 100 previous exchanges (capped at 15,000 tokens). Users can start new conversation threads, view conversation history in a sidebar, and conversations are persisted in the PostgreSQL database for long-term access. The chat history is formatted and included in every LLM prompt to enable coherent multi-turn conversations.

4.1.3 Streaming Responses (FR-003)

AI responses shall appear in real-time as they are generated, using Server-Sent Events (SSE) for server-to-client streaming. The first token should appear within 1.5 seconds (TTFT). The streaming protocol distinguishes between thought tokens (agent reasoning), content tokens (final answer), source events (retrieved documents), and status events (search progress updates). Users can observe the AI's reasoning process in real-time.

4.1.4 Multi-Format Document Upload (FR-004)

The system shall support document upload in the following formats: PDF (via PyMuPDF), DOCX (via python-docx), CSV and XLSX (via pandas/openpyxl), TXT, and Markdown. Maximum file size is 50 MB per file. The upload process includes file validation, checksum computation (SHA-256), metadata extraction, and queuing for asynchronous processing via Django-Q2.

4.1.5 Automatic Document Processing (FR-005)

Uploaded documents shall be automatically processed through a multi-stage pipeline: text extraction with format-specific parsers, intelligent chunking (800 tokens with 200-token overlap), embedding generation using Gemini Embedding 2.0 (3,072-dimensional vectors), and vector indexing in Qdrant with metadata for

namespace isolation. Processing status is tracked through a state machine (Pending → Processing → Indexed | Error).

4.1.6 Document Organization (FR-006)

The system shall provide collection-based organization for documents, allowing users to create named collections with custom colors. A tagging system enables flexible categorization across collections. Users can search within their document library, and soft deletion with recovery is supported. Each user's documents are isolated through namespace-based filtering in the vector store.

4.1.7 Source Citation (FR-007)

Every AI-generated answer shall include references to the source documents from which the information was retrieved. Citations include the document name, chunk index, relevance score, and a content preview of the source context. This ensures complete transparency and allows users to verify the accuracy of responses.

4.1.8 Authentication and Authorization (FR-010)

The system implements JWT-based authentication with access tokens (15-minute validity) and refresh tokens (7-day validity). Three user roles are defined with distinct permission levels:

ROLE	PERMISSIONS
ADMIN	FULL SYSTEM ACCESS — MANAGE USERS, VIEW ALL ANALYTICS, CRUD ALL RESOURCES
EDITOR	UPLOAD DOCUMENTS, CHAT, MANAGE COLLECTIONS, VIEW DASHBOARD STATISTICS
VIEWER	CHAT ONLY — READ-ONLY DOCUMENT ACCESS, VIEW OWN QUERY HISTORY

Table 4.3: User Roles and Permissions

4.1.9 Analytics Dashboard (FR-012)

An administrative dashboard shall provide system-wide usage analytics, including total queries and active users over configurable time periods, query volume trends displayed on interactive charts, document processing statistics and storage utilization, confidence score distributions across queries, and exportable reports in CSV format. The dashboard enables administrators to monitor system health, identify usage patterns, and make data-driven decisions about system configuration.

4.2 Non-Functional Requirements

4.2 Non-Functional Requirements

CATEGORY	REQUIREMENT	TARGET METRIC
PERFORMANCE	TIME TO FIRST TOKEN (TTFT)	< 1.5 SECONDS
PERFORMANCE	TOTAL RESPONSE TIME (500 TOKENS)	< 5 SECONDS
PERFORMANCE	VECTOR SEARCH LATENCY	< 500 MILLISECONDS
PERFORMANCE	DOCUMENT INDEXING (10 MB)	< 2 MINUTES
SCALABILITY	CONCURRENT USERS	50+ SIMULTANEOUS SESSIONS
SCALABILITY	DOCUMENT CORPUS	1M+ DOCUMENT CHUNKS
SCALABILITY	STORAGE	100 GB+ DOCUMENT REPOSITORY
RELIABILITY	SYSTEM UPTIME	99.5%
RELIABILITY	ERROR RECOVERY	AUTO-RETRY WITH EXPONENTIAL BACKOFF
RELIABILITY	DATA BACKUP	DAILY AUTOMATED BACKUPS
SECURITY	AUTHENTICATION	JWT WITH REFRESH ROTATION
SECURITY	RATE LIMITING	100 REQUESTS/MIN (USER), 15/MIN (CHAT)
SECURITY	DATA PROTECTION	TLS 1.3, HSTS, CSP HEADERS
MAINTAINABILITY	CODE QUALITY	STRUCTURED LOGGING, API DOCS
MAINTAINABILITY	CONTAINERIZATION	DOCKER COMPOSE FOR ALL SERVICES

Table 4.2: Non-Functional Requirements

CHAPTER 5: SYSTEM DESIGN

This chapter presents the detailed system design of the AI-Powered RAG Chatbot, covering the overall system architecture, process flows, data flows, database design, and UML diagrams. The design follows established software engineering principles including separation of concerns, modularity, scalability, and security-by-design.

5.1 System Architecture

5.1 System Architecture

The system follows a modern microservices-inspired architecture with clear separation between the client tier, API gateway, application services, RAG engine, and data storage layers. The architecture is designed for horizontal scalability, with stateless backend services that can be replicated behind a load balancer.

The architecture consists of five primary tiers:

- CLIENT TIER: REACT 19 SINGLE PAGE APPLICATION (SPA) WITH TYPESCRIPT, COMMUNICATING WITH THE BACKEND EXCLUSIVELY VIA REST API CALLS AND SSE STREAMS.
- API GATEWAY: DJANGO REST FRAMEWORK WITH GUNICORN, HANDLING AUTHENTICATION (JWT), RATE LIMITING, REQUEST ROUTING, AND CORS MANAGEMENT.
- APPLICATION SERVICES: MODULAR DJANGO APPS FOR CHAT, DOCUMENTS, USERS, ANALYTICS, AND RAG, EACH WITH ITS OWN MODELS, SERIALIZERS, VIEWS, AND URL CONFIGURATIONS.
- RAG ENGINE: LANGCHAIN-BASED PIPELINE ORCHESTRATING QUERY PROCESSING, VECTOR SEARCH, CONTEXT ASSEMBLY, AND LLM INTERACTION THROUGH AN AGENTIC REACT LOOP.
- DATA STORAGE: POSTGRESQL 15 FOR RELATIONAL DATA (USERS, SESSIONS, DOCUMENTS), QDRANT FOR VECTOR EMBEDDINGS (3072-DIMENSIONAL), AND REDIS 7 FOR CACHING AND TASK QUEUE MANAGEMENT.

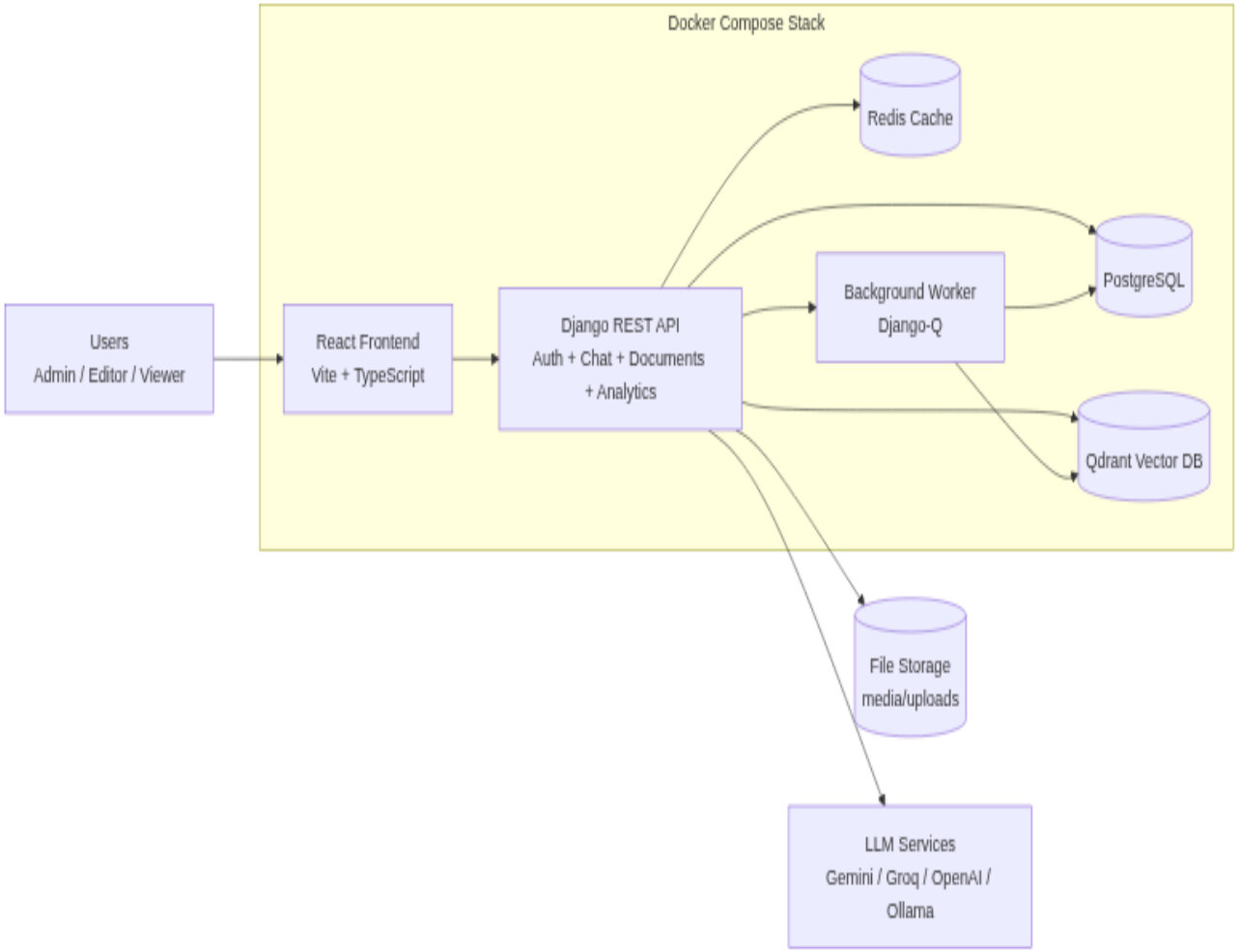

Figure 5.1: System Architecture Diagram

5.2 Flowcharts

The following flowchart illustrates the primary user interaction flow within the AI-Powered RAG Chatbot system. The flow begins with user authentication, proceeds through document management and chat interactions, and concludes with response delivery.

The application follows a systematic flow:

1. User visits the landing page and either registers a new account or logs in with existing credentials.

2. Upon successful authentication, the user is redirected to the main dashboard showing usage analytics and recent activity.

3. The user can navigate to the Document Management module to upload and organize documents for RAG processing.

4. Uploaded documents enter an asynchronous processing pipeline: text extraction → chunking → embedding → vector indexing.

5. The user navigates to the Chat interface to ask questions about their uploaded documents.

6. User queries trigger the agentic RAG pipeline: query rewriting → vector search → context assembly → LLM generation.

7. The AI response is streamed back in real-time via SSE, with source citations and confidence scores.

8. The user can review query history, save frequent searches, export conversations, and provide feedback on response quality.

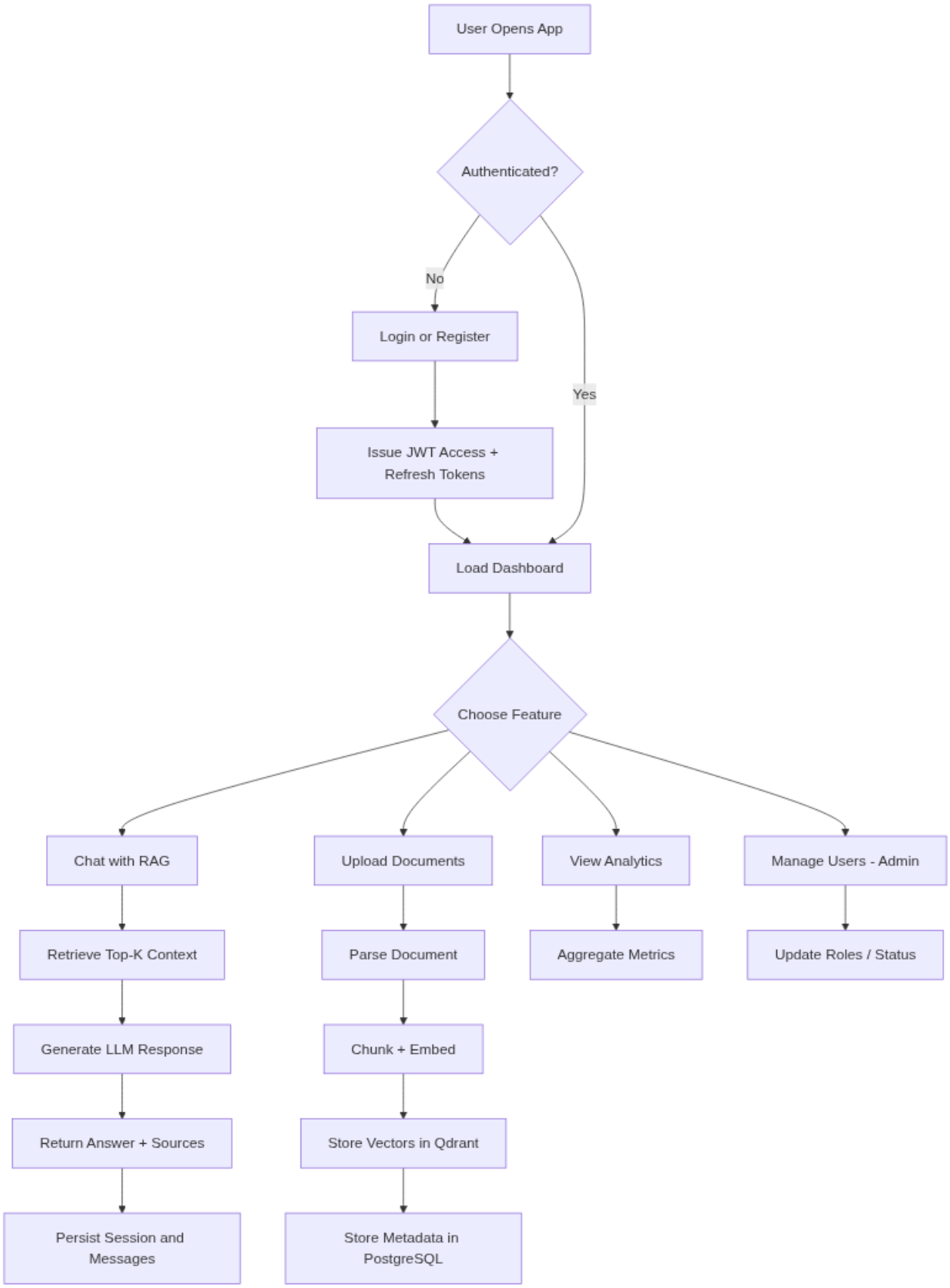

Figure 5.2: Application Flowchart

5.3 Data Flow Diagrams (DFD)

The Data Flow Diagram illustrates how data moves through the system from external entities (Users, Google Gemini API) through internal processes (Authentication, Document Processing, RAG Pipeline, Analytics) to data stores (PostgreSQL, Qdrant, Redis, File Storage).

Key data flows in the system include:

1. User → Authentication Module: Login credentials (email, password) for JWT token generation.

2. User → Document Upload: Raw document files (PDF, DOCX, CSV, etc.) with metadata.

3. Document Upload → Processing Pipeline: Validated files queued for asynchronous text extraction and chunking.

4. Processing Pipeline → Embedding Service: Text chunks sent to Google Gemini Embedding API for 3072-dimensional vector generation.

5. Embedding Service → Qdrant Vector Store: Embedded vectors stored with metadata (user_id, document_id, collection_id) for namespace isolation.

6. User → Chat Module: Natural language queries with optional collection scope and file attachments.

7. Chat Module → RAG Engine: Query text with conversation history for agentic processing.

8. RAG Engine → Qdrant: Semantic similarity search with metadata filters for user-scoped retrieval.

9. RAG Engine → Google Gemini LLM: Assembled prompt with retrieved context for response generation.

10. RAG Engine → User (via SSE): Streamed tokens (thoughts, content, sources, status) in real-time.

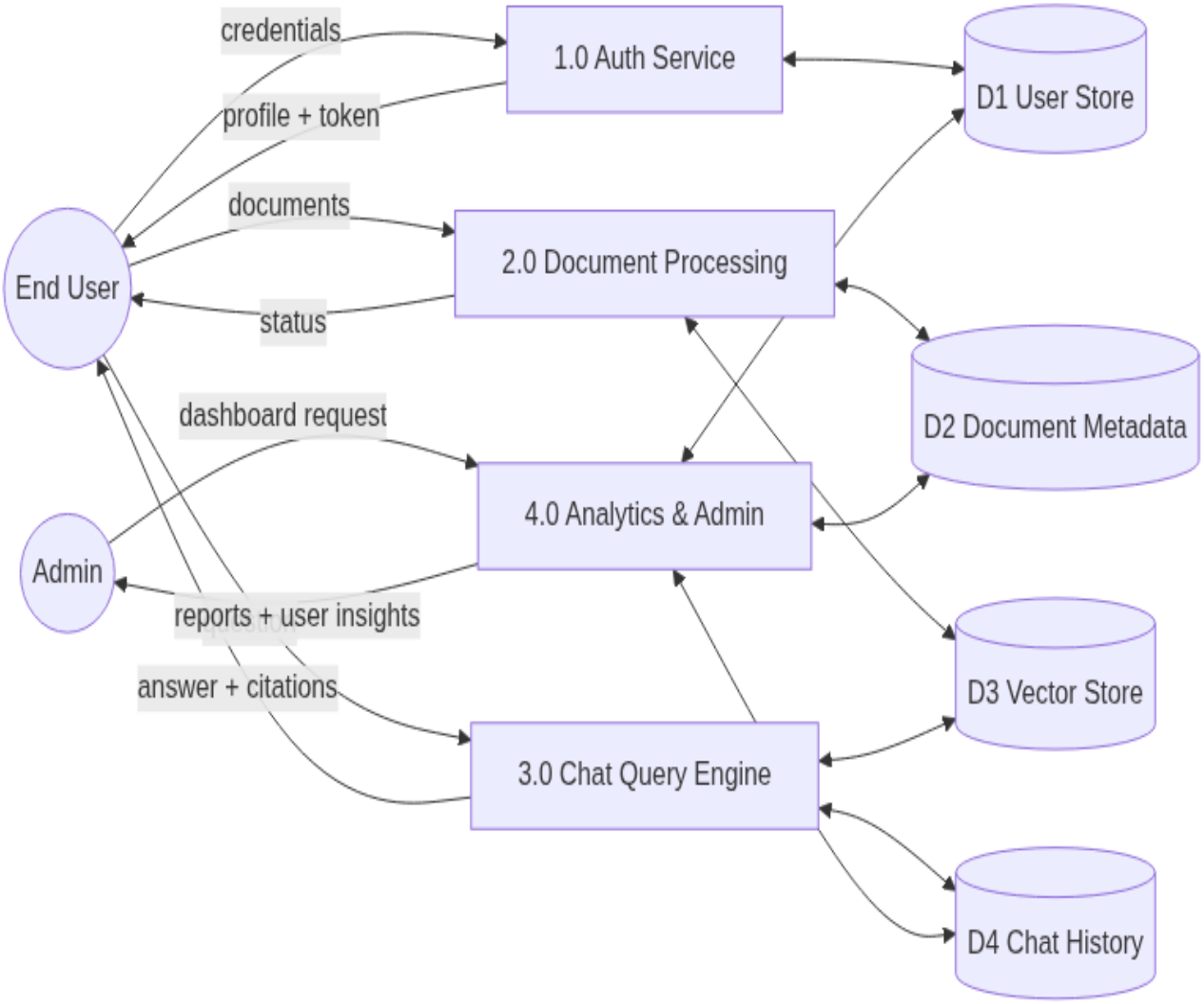

Figure 5.3: Data Flow Diagram (DFD) – Level 1

5.4 Database Design (ERD)

The system uses PostgreSQL 15 as the primary relational database for storing structured data. The database schema follows Django's ORM conventions with explicit table names, UUID primary keys, and timestamp-based audit trails. The following entities constitute the core data model:

- User: custom user model with email-based authentication, uuid primary key, role-based access control (admin/editor/viewer), and standard audit fields.
- Chatsession: represents a conversation thread belonging to a user, with auto-generated titles and archive capability.
- Message: individual messages within a chat session, storing role (user/assistant/system), content, token usage, latency metrics, source citations (json), and confidence scores.
- Messageattachment: file attachments linked to messages for multimodal rag queries.
- Messagefeedback: thumbs up/down feedback on ai responses with optional text comments.
- Savedsearch: user-bookmarked queries with collection scope for quick re-execution.
- Document: uploaded document metadata including filename, file path, mime type, sha-256 checksum, processing status (pending/processing/indexed/error), and collection assignment.
- Documentchunk: text chunks extracted from documents, mirroring qdrant entries with chunk index, content, token count, and embedding id.
- Collection: named groupings for document organization with owner-based isolation.
- Tag: flexible categorization labels with many-to-many relationship to documents.

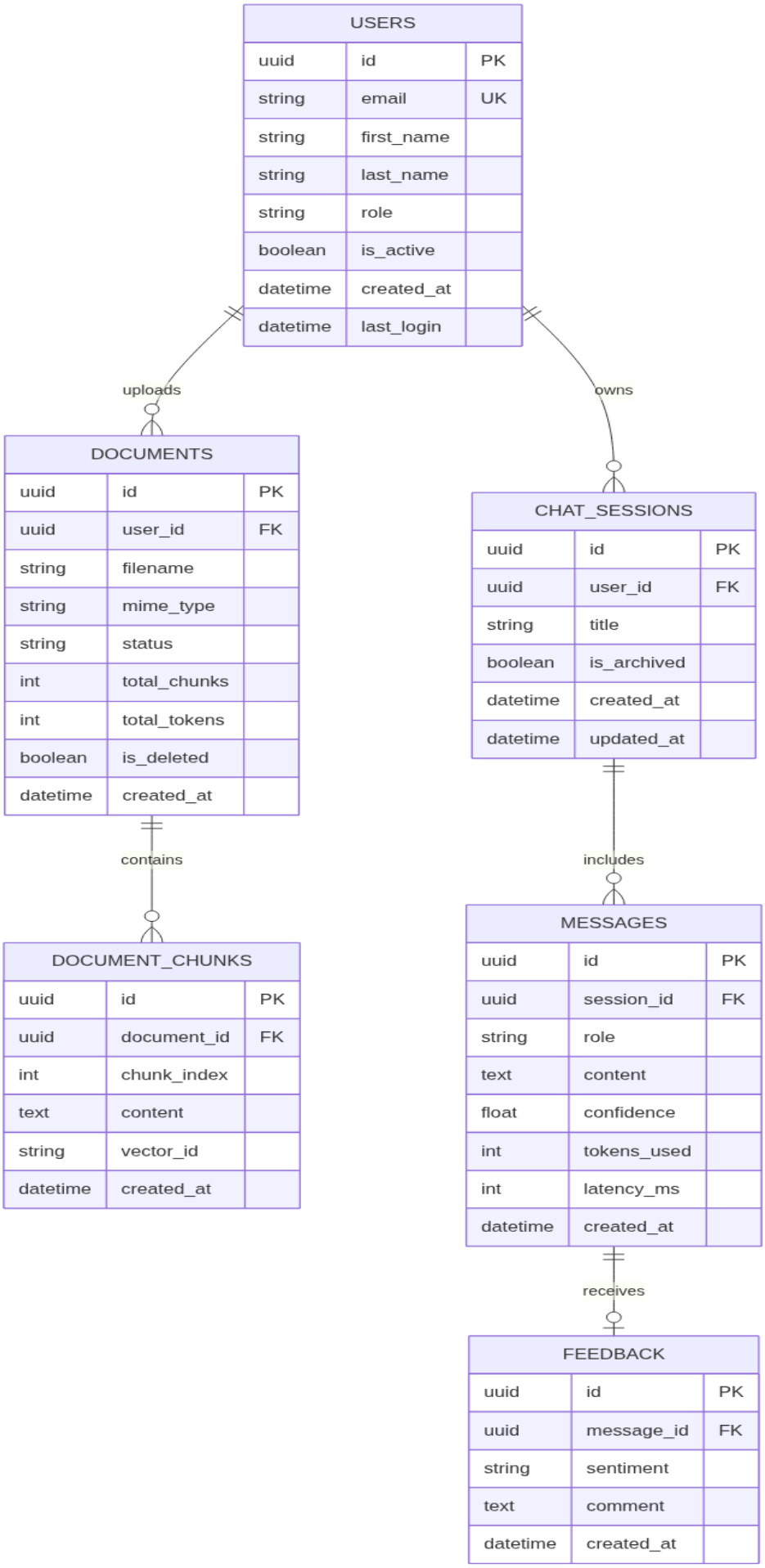

Figure 5.4: Database Design – Entity Relationship Diagram

5.5 UML Design

The Unified Modeling Language (UML) diagrams provide a comprehensive visual representation of the system's structure and behavior. The following UML overview shows the relationships between the major system components and their interactions.

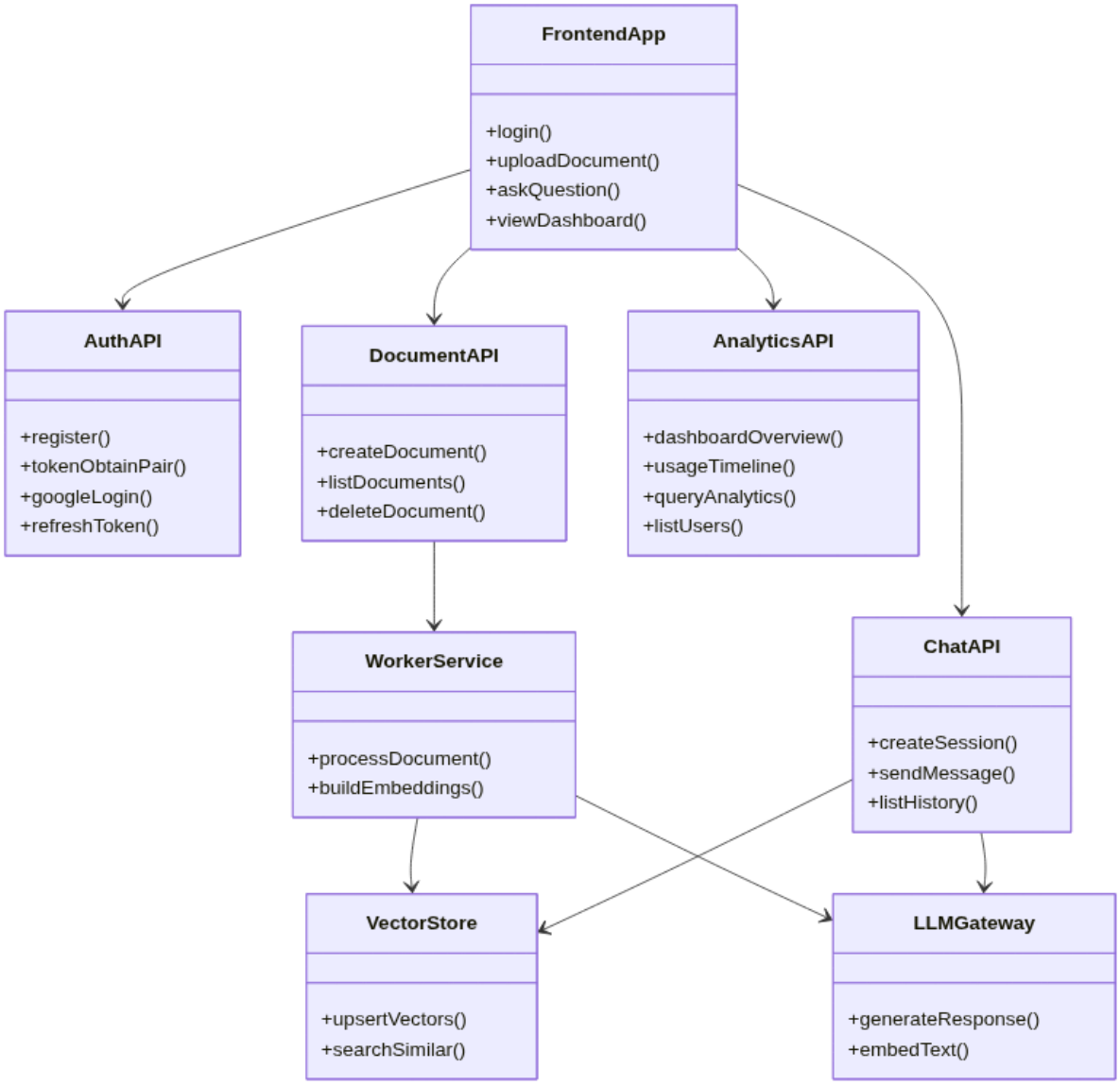

Figure 5.5: UML Design Overview

5.6 Use Case Diagrams

The use case diagram identifies the primary actors (Viewer, Editor, Admin, and External Systems) and the actions they can perform within the system. Key use cases include:

- VIEWER: REGISTER/LOGIN, ASK QUESTIONS, VIEW CHAT HISTORY, SEARCH SAVED QUERIES, EXPORT CHAT, PROVIDE FEEDBACK.
- EDITOR: ALL VIEWER CAPABILITIES PLUS UPLOAD DOCUMENTS, CREATE COLLECTIONS, MANAGE TAGS, VIEW DASHBOARD.
- ADMIN: ALL EDITOR CAPABILITIES PLUS MANAGE USERS, VIEW SYSTEM ANALYTICS, CONFIGURE RAG SETTINGS, MONITOR SYSTEM HEALTH.
- GOOGLE GEMINI API (EXTERNAL): PROVIDE LLM INFERENCE, GENERATE EMBEDDINGS.
- QDRANT (EXTERNAL): STORE/RETRIEVE VECTOR EMBEDDINGS, PERFORM SIMILARITY SEARCH.

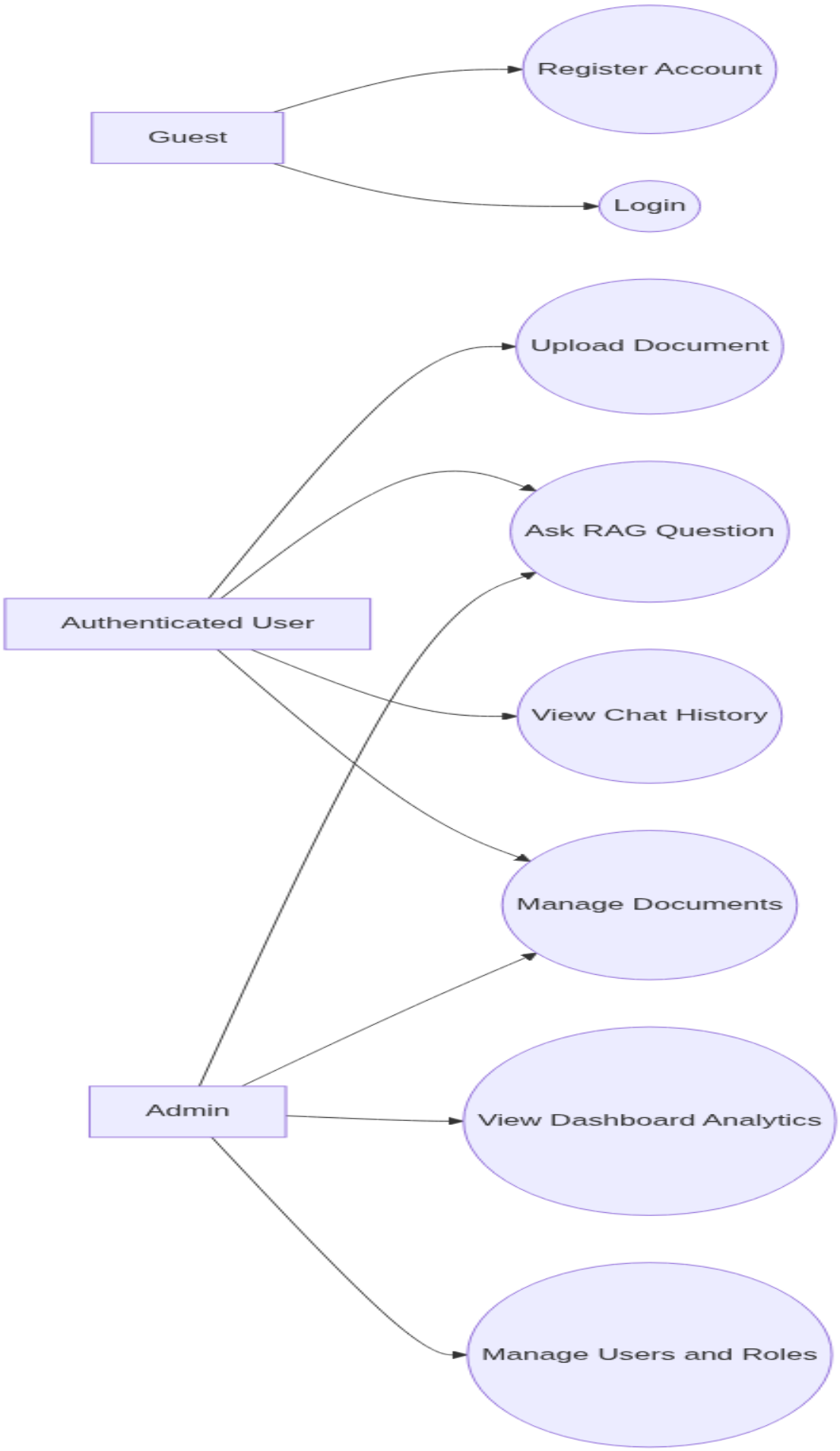

Figure 5.6: Use Case Diagram

5.7 Sequence Diagrams

The sequence diagram illustrates the temporal interaction between components during a typical RAG query flow. The sequence proceeds through the following steps:

1. The User sends a natural language query through the React Frontend.

2. The Frontend forwards the query to the Django Backend via a POST request to /api/chat/sessions/{id}/stream/.

3. The Backend authenticates the request via JWT, checks rate limits, and initializes the RAG pipeline.

4. The RAG Engine invokes the ReAct Agent, which formulates a Thought and decides on an Action (search).

5. The Agent sends the optimized search query to Qdrant for vector similarity search with user-scoped filters.

6. Qdrant returns relevant document chunks ranked by cosine similarity.

7. The Agent observes the results, evaluates sufficiency, and may iterate (up to 3 times).

8. The Agent formulates a Final Answer grounded in the retrieved context.

9. The response is streamed back to the Frontend via SSE with token-by-token delivery.

10. The Frontend renders the response in real-time with source citations and confidence indicators.

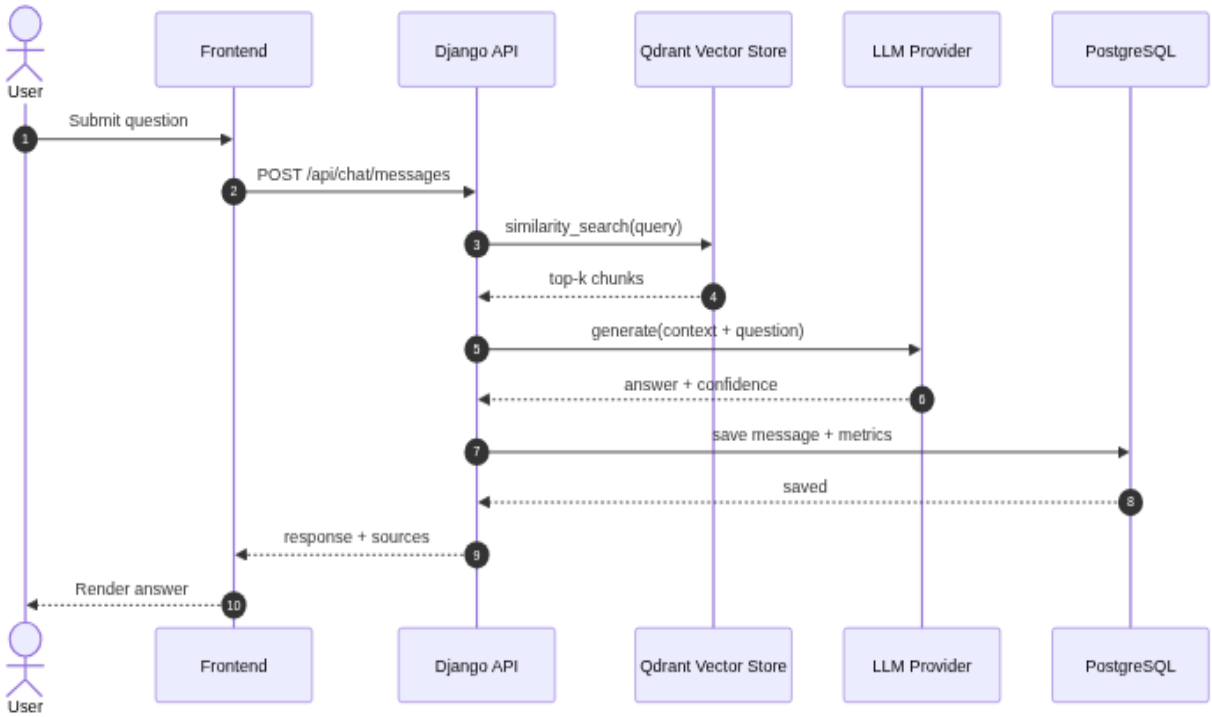

Figure 5.7: Sequence Diagram

5.8 Class Diagrams

The class diagram represents the object-oriented structure of the backend application, showing the key Django model classes, their attributes, methods, and relationships. The diagram highlights the inheritance from TimeStampedModel (providing created_at and updated_at fields) and the foreign key relationships between models.

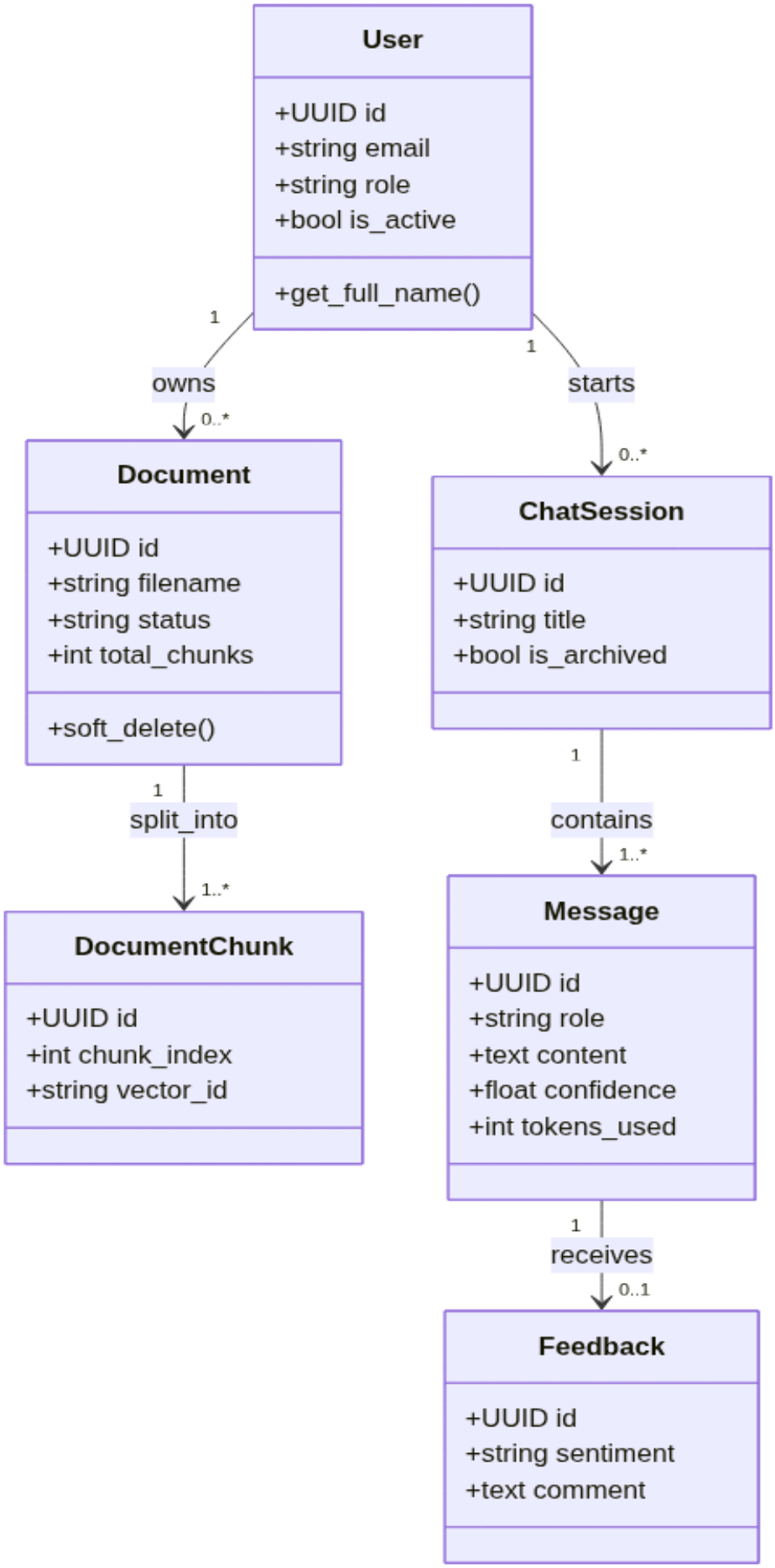

Figure 5.8: Class Diagram

5.9 Activity Diagrams

The activity diagram models the workflow of the document processing pipeline, from initial upload through text extraction, chunking, embedding, and vector indexing. It shows the parallel and sequential activities, decision points (format detection, error handling), and the state transitions of the Document model (Pending → Processing → Indexed | Error).

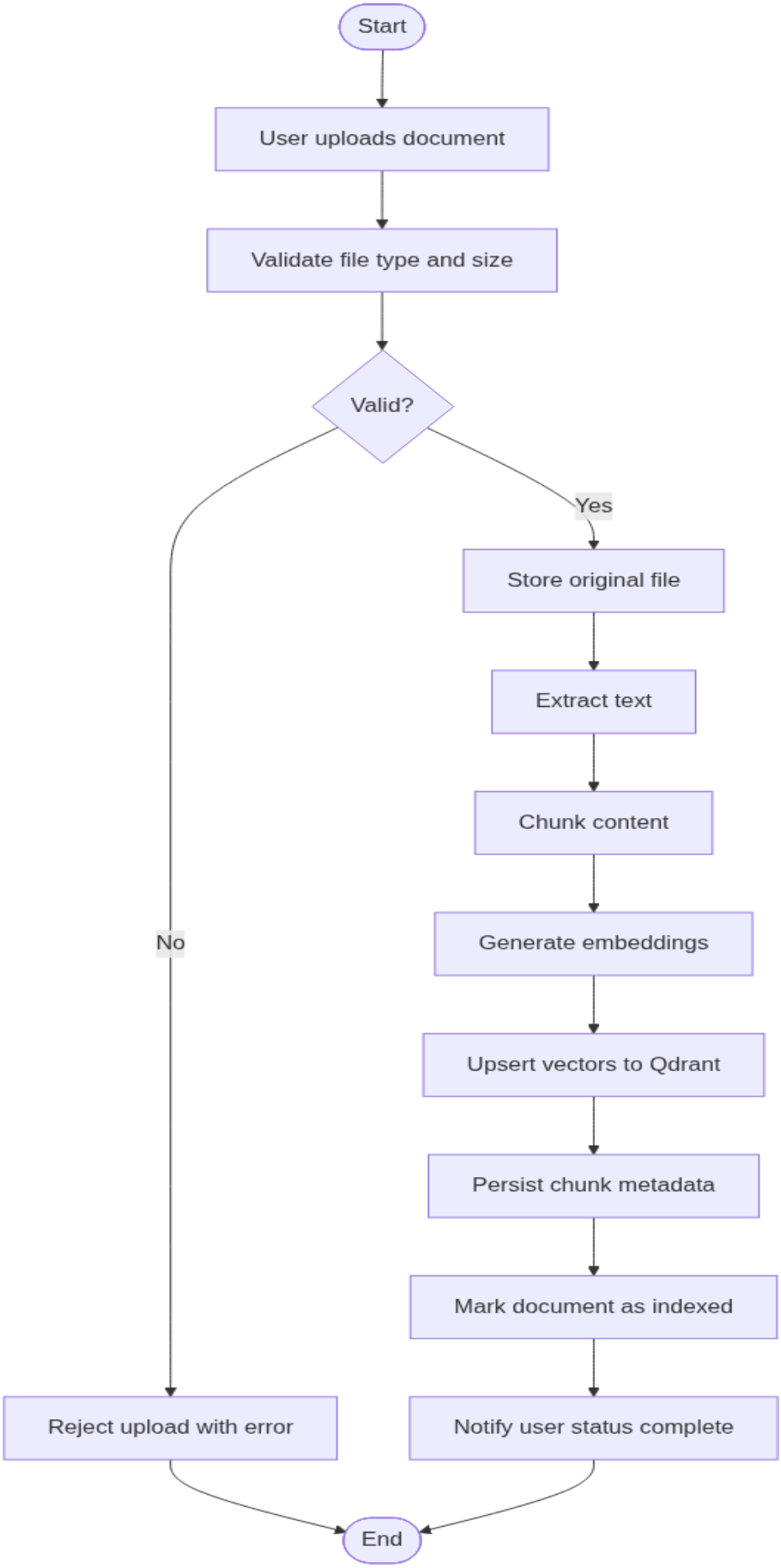

Figure 5.9: Activity Diagram

CHAPTER 6: IMPLEMENTATION

This chapter presents the implementation details of the AI-Powered RAG Chatbot, including key source code excerpts from the backend and frontend, the RAG pipeline architecture, and screenshots of the running application. The implementation follows best practices in software engineering including modular design, clean code principles, comprehensive error handling, and structured logging.

6.1 Backend Implementation

6.1 Backend Implementation

6.1.1 User Model (apps/users/models.py)

The custom User model extends Django's AbstractBaseUser with email-based authentication and role-based access control:

```
class UserRole(models.TextChoices):
    ADMIN = "admin", "Admin"
    EDITOR = "editor", "Editor"
    VIEWER = "viewer", "Viewer"

class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField(unique=True, max_length=255)
    first_name = models.CharField(max_length=150, blank=True)
    last_name = models.CharField(max_length=150, blank=True)
    role = models.CharField(max_length=10, choices=UserRole.choices,
                            default=UserRole.EDITOR, db_index=True)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["first_name", "last_name"]
```

6.1.2 Chat Session and Message Models (apps/chat/models.py)

The chat models implement a hierarchical structure: ChatSession → Message → MessageAttachment, with JSON fields for source citations:

```
class ChatSession(TimeStampedModel):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
                             related_name="chat_sessions")
    title = models.CharField(max_length=255, default="New Chat")
    is_archived = models.BooleanField(default=False, db_index=True)


class Message(TimeStampedModel):
    session = models.ForeignKey(ChatSession, on_delete=models.CASCADE,
                                related_name="messages")
    role = models.CharField(max_length=10, choices=MessageRole.choices)
```

```
    content = models.TextField()
    tokens_used = models.IntegerField(default=0)
    latency_ms = models.IntegerField(default=0)
    sources = models.JSONField(default=list, blank=True)
    confidence_score = models.FloatField(null=True, blank=True)
```

6.1.3 Document Model (apps/documents/models.py)

The Document model tracks the complete lifecycle of uploaded files with status tracking and checksum verification:

```
class Document(TimeStampedModel):
    filename = models.CharField(max_length=255)
    original_name = models.CharField(max_length=255)
    file_path = models.CharField(max_length=500)
    file_size = models.BigIntegerField()
    mime_type = models.CharField(max_length=100)
    checksum = models.CharField(max_length=64)  # SHA256
    status = models.CharField(max_length=20, choices=DocumentStatus.choices,
                            default=DocumentStatus.PENDING, db_index=True)
    error_message = models.TextField(blank=True)
    metadata = models.JSONField(default=dict, blank=True)
    collection = models.ForeignKey(Collection, on_delete=models.SET_NULL,
                                   null=True, blank=True)
    tags = models.ManyToManyField(Tag, blank=True, related_name="documents")
    uploaded_by = models.ForeignKey(settings.AUTH_USER_MODEL,
                                    on_delete=models.CASCADE)
    indexed_at = models.DateTimeField(null=True, blank=True)
    is_deleted = models.BooleanField(default=False, db_index=True)
```

6.1.4 Docker Compose Configuration (docker-compose.yml)

The entire application stack is containerized using Docker Compose with five services:

```
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: raguser
      POSTGRES_PASSWORD: ragpassword
      POSTGRES_DB: ragchatbot
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U raguser -d ragchatbot"]


  redis:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
```

```
  qdrant:
    image: qdrant/qdrant:latest
    ports:
      - "6333:6333"


  backend:
    build:
      dockerfile: docker/backend.Dockerfile
    command: >
      sh -c "python manage.py migrate --noinput &&
             python manage.py runserver 0.0.0.0:8000"
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_healthy }
      qdrant: { condition: service_started }


  frontend:
    build:
      dockerfile: docker/frontend.Dockerfile
    ports:
      - "5173:5173"
```

6.2 Frontend Implementation

The frontend is built as a single-page application using React 19 with TypeScript, leveraging Vite for development builds and Tailwind CSS for utility-first styling.

6.2.1 Application Router (App.tsx)

The React application uses React Router v7 with lazy-loaded route components for optimal performance:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const ChatPage = lazy(() => import('./pages/chat'));
const DocumentsPage = lazy(() => import('./pages/documents'));
const DashboardPage = lazy(() => import('./pages/dashboard'));
const HistoryPage = lazy(() => import('./pages/history'));

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <Suspense fallback={<LoadingSpinner />}>
          <Routes>
            <Route path="/" element={<LandingPage />} />
            <Route path="/login" element={<LoginPage />} />
            <Route path="/register" element={<RegisterPage />} />
            <Route path="/chat" element={<ProtectedRoute><ChatPage /></ProtectedRoute>} />
            <Route path="/documents" element={<ProtectedRoute><DocumentsPage /></ProtectedRoute>}
/>
            <Route path="/dashboard" element={<AdminRoute><DashboardPage /></AdminRoute>} />
          </Routes>
        </Suspense>
      </BrowserRouter>
    </QueryClientProvider>
  );
}
```

6.2.2 State Management with Zustand

The application uses Zustand for lightweight, performant client-side state management. Separate stores handle authentication state, chat state, and document state, keeping concerns cleanly separated.

6.2.3 API Client with Axios

A centralized Axios instance handles all API communication with automatic JWT token injection, refresh token rotation on 401 responses, and structured error handling.

6.3 RAG Pipeline Implementation

The core RAG pipeline is the most critical component of the system, implementing an agentic architecture with multi-step reasoning.

6.3.1 Query Rewriting Module (apps/rag/query.py)

User queries are preprocessed through an intelligent rewriting module that optimizes them for retrieval:

```
def rewrite_query(user_query: str, chat_history: str = "") -> dict:
    """Rewrite a user query for better retrieval.
    Returns dict with keys: rewritten, keywords, intent"""
    llm = get_llm(streaming=False)
    prompt = QUERY_REWRITE_PROMPT.format(
        query=user_query,
        chat_history=chat_history or "No previous conversation.",
    )
    response = llm.invoke([HumanMessage(content=prompt)])
    result = json.loads(str(response.content).strip())
    return {
        "rewritten": result.get("rewritten", user_query),
        "keywords": result.get("keywords", []),
        "intent": result.get("intent", "general"),
    }
```

6.3.2 Vector Store Integration (apps/rag/services/vector_store.py)

The vector store module manages all interactions with Qdrant, including collection creation, document indexing, similarity search with metadata filtering, and document deletion:

```
def similarity_search(query, k=None, filter_dict=None):
    """Search for similar documents with namespace segregation."""
    search_k = k if k else int(settings.RAG_CONFIG["top_k"])
    threshold = settings.RAG_CONFIG["similarity_threshold"]
    vectorstore = get_vectorstore()

    # Build Qdrant filter for user-scoped retrieval
    qdrant_filter = None
    if filter_dict:
        must_conditions = []
        for key, value in filter_dict.items():
            must_conditions.append(
                rest.FieldCondition(
                    key=f"metadata.{key}",
                    match=rest.MatchValue(value=value)
                )
            )
        qdrant_filter = rest.Filter(must=must_conditions)
```

```
    results = vectorstore.similarity_search_with_relevance_scores(
        query=query, k=search_k, filter=qdrant_filter,
    )
    return [(doc, score) for doc, score in results if score >= threshold]
```

6.3.3 Agentic RAG Chain with ReAct Loop (apps/rag/services/rag_chain.py)

The core streaming function implements a ReAct agent loop that can iterate up to 3 times, performing search actions and evaluating observations before formulating a final answer:

```
def stream_chat_response(session, user_message, collection="default"):
    # 1. Save user message and auto-generate session title
    msg = Message.objects.create(session=session, role=MessageRole.USER,
                                 content=user_message)

    # 2. Build chat history (up to 15,000 tokens)
    chat_history = _build_chat_history(session)

    # 3. Initialize ReAct Agent
    system_content = AGENT_SYSTEM_PROMPT.format(chat_history=chat_history)
    current_messages = [
        SystemMessage(content=system_content),
        HumanMessage(content=f"Question: {user_message}")
    ]

    # 4. Agent Loop (max 3 iterations)
    for i in range(3):
        stream = _call_llm_with_retry(llm, current_messages)
        for chunk in stream:
            if "Final Answer:" in content:
                yield json.dumps({'type': 'token', 'content': token})
            elif 'Action: search("' in content:
                # Execute search action
                search_results = similarity_search(query=search_query,
                    k=settings.RAG_CONFIG["top_k"], filter_dict=filter_dict)
                yield json.dumps({'type': 'sources', 'data': sources})
            else:
                yield json.dumps({'type': 'thought', 'content': token})

    # 5. Save assistant message with confidence score
    Message.objects.create(session=session, role=MessageRole.ASSISTANT,
        content=final_answer, sources=list(unique_sources),
        confidence_score=0.9)
```

6.3.4 Embedding Service (apps/rag/services/embeddings.py)

The embedding module supports multiple providers (Gemini, OpenAI, Mock) with automatic detection and exponential backoff retry logic for rate limiting:

```
class GeminiEmbeddings(Embeddings):
    """Google Gemini embeddings - 3072-dimensional vectors."""
    def embed_documents(self, texts):
        all_embeddings = []
        batch_size = 90
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i + batch_size]
            for attempt in range(8):  # Exponential backoff
                try:
                    result = client.models.embed_content(
                        model=self.model, contents=batch,
                        config=types.EmbedContentConfig(
                            task_type="RETRIEVAL_DOCUMENT"))
                    all_embeddings.extend(
                        [e.values for e in result.embeddings])
                    time.sleep(2.5)  # Rate limit compliance
                    break
                except Exception as e:
                    if is_rate_limit_error(e):
                        wait = min((2 ** attempt) * 10, 60)
                        time.sleep(wait)
                    else: raise
        return all_embeddings
```

6.4 Output Screenshots

The following screenshots demonstrate the fully functional AI-Powered RAG Chatbot application running in a development environment.

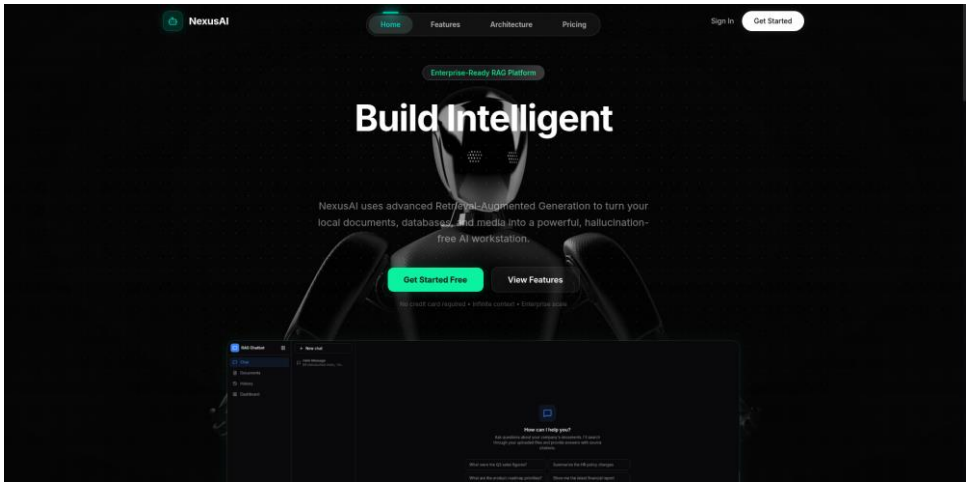

Figure 6.1: Landing Page – Hero Section with 3D Interactive Scene

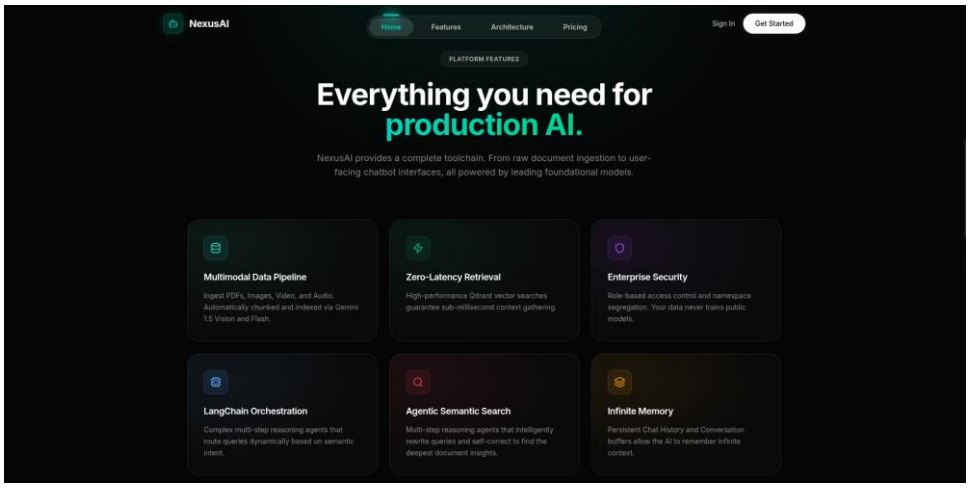

Figure 6.2: Landing Page – Feature Highlights Section

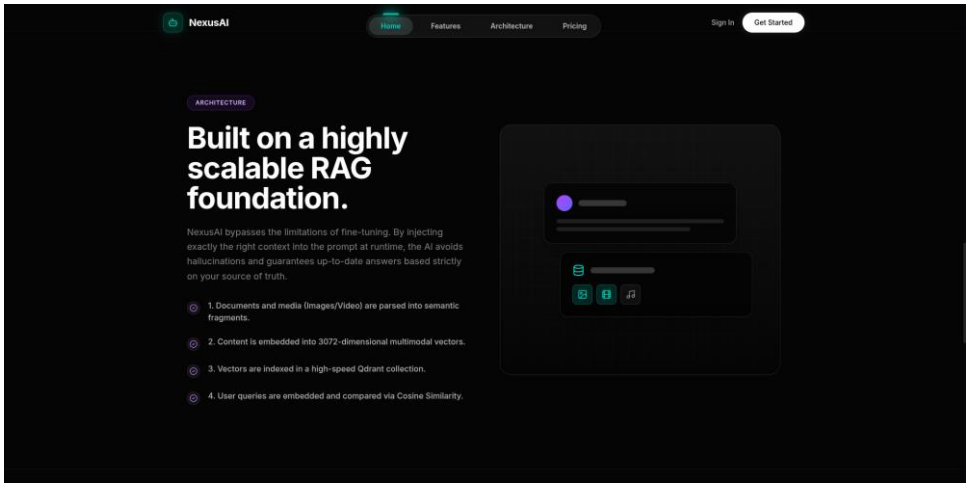

Figure 6.3: Landing Page – Call to Action and Footer

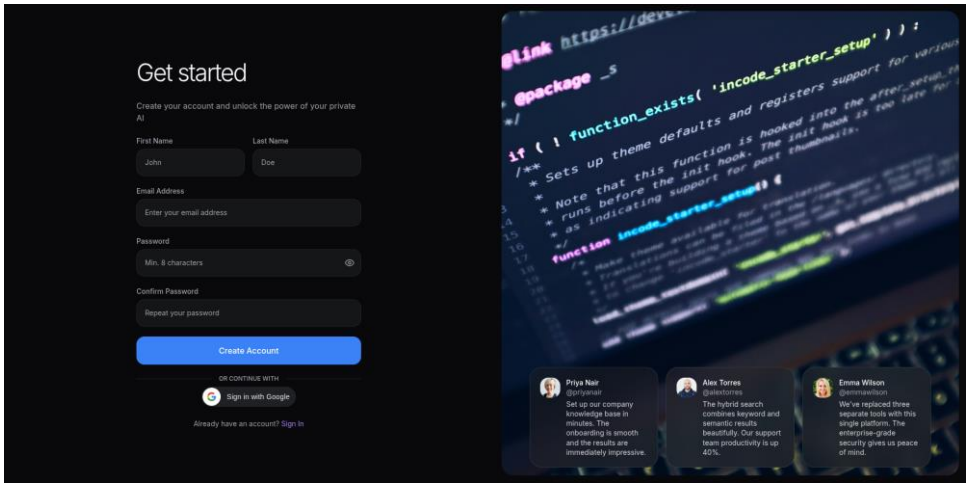

Figure 6.4: User Registration Page

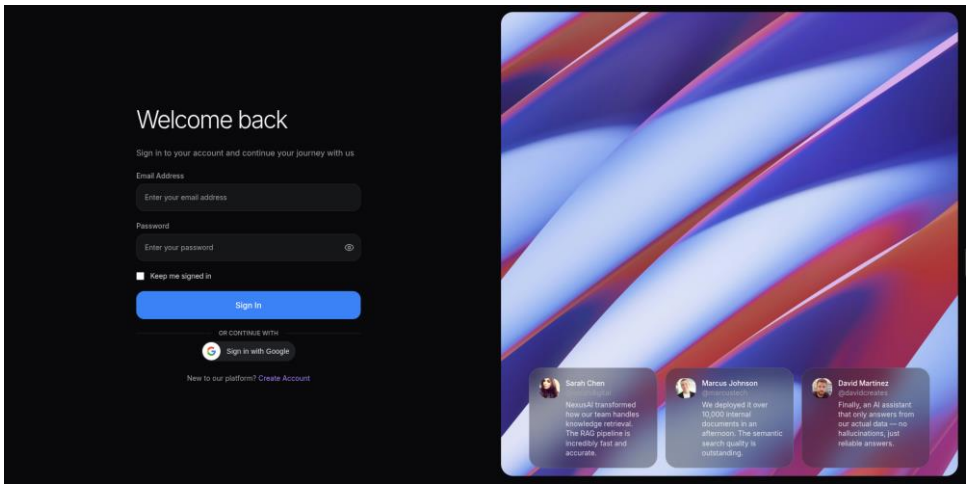

Figure 6.5: User Login Page

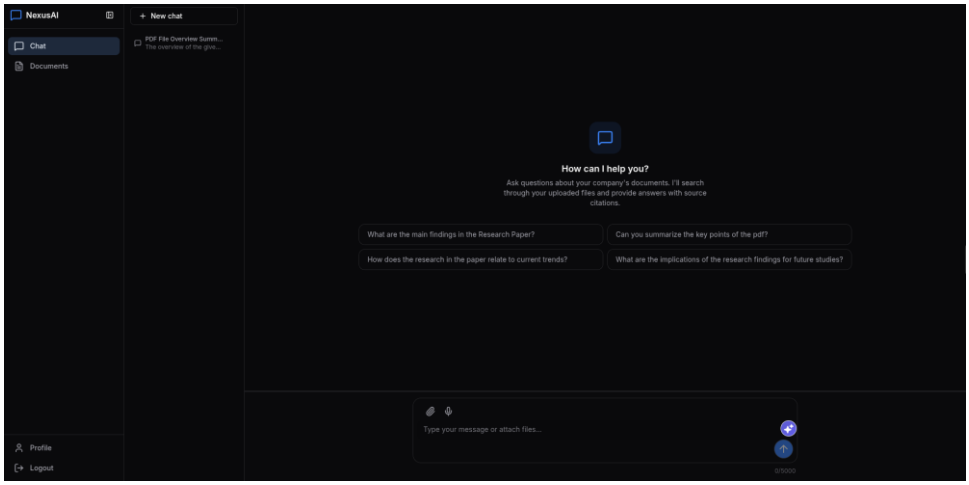

Figure 6.6: Chat Interface – AI-Powered Conversation with Source Citations

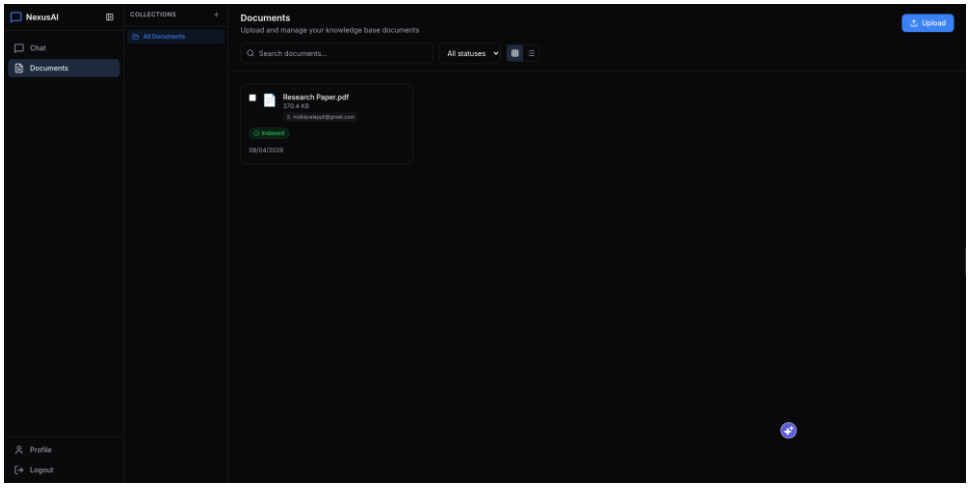

Figure 6.7: Document Upload and Management Interface

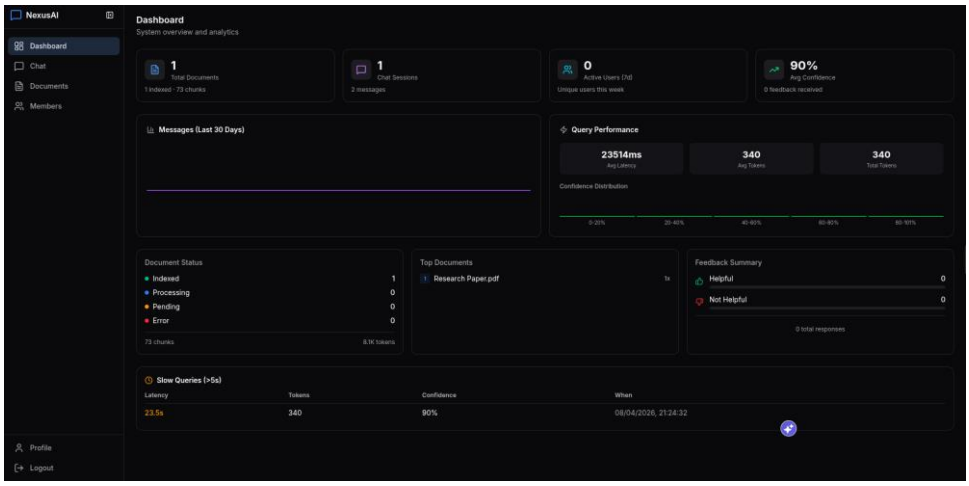

Figure 6.8: Admin Dashboard – Usage Analytics and System Metrics

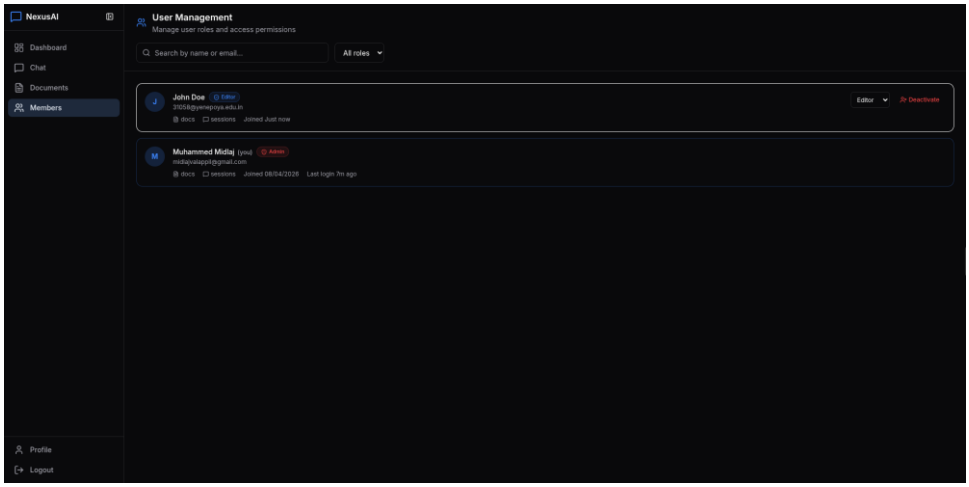

Figure 6.9: User Management Panel – Admin Controls

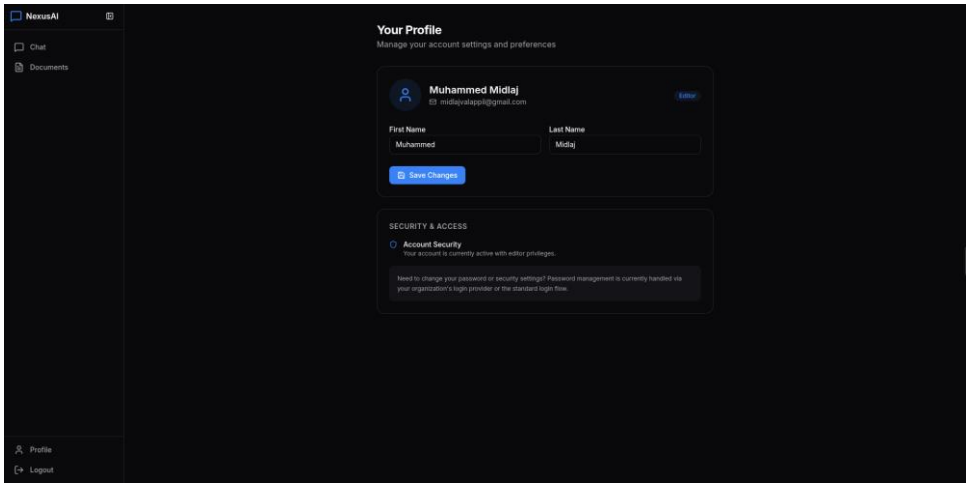

Figure 6.10: User Profile Section

CHAPTER 7: TEST CASES

This chapter presents the testing strategy employed to ensure the reliability, correctness, and robustness of the AI-Powered RAG Chatbot system. Testing was conducted at multiple levels: unit testing for individual modules, integration testing for inter-module interactions, and system testing for end-to-end functionality validation.

7.1 Unit Testing

7.1 Unit Testing

Unit tests verify the correctness of individual components in isolation. The backend uses pytest with Django's test framework, while the frontend uses Vitest with React Testing Library. The following table presents key unit test cases:

TEST CASE ID	MODULE	TEST DESCRIPTION	INPUT	EXPECTED OUTPUT	RESULT
UT-001	USER REGISTRATION	REGISTER WITH VALID EMAIL AND PASSWORD	EMAIL: TEST@EXAMPLE.COM, PASSWORD: SECUREPASS123!	USER CREATED SUCCESSFULLY, JWT TOKENS RETURNED	PASS
UT-002	USER REGISTRATION	REGISTER WITH DUPLICATE EMAIL	EMAIL: EXISTING@EXAMPLE.COM	400 BAD REQUEST: EMAIL ALREADY EXISTS	PASS
UT-003	USER LOGIN	LOGIN WITH CORRECT CREDENTIALS	VALID EMAIL AND PASSWORD	200 OK: ACCESS AND REFRESH TOKENS RETURNED	PASS
UT-004	USER LOGIN	LOGIN WITH INCORRECT PASSWORD	VALID EMAIL, WRONG PASSWORD	401 UNAUTHORIZED: INVALID CREDENTIALS	PASS
UT-005	DOCUMENT UPLOAD	UPLOAD VALID PDF FILE	PDF FILE (< 50 MB)	DOCUMENT CREATED WITH STATUS 'PENDING'	PASS

UT-006	DOCUMENT UPLOAD	UPLOAD UNSUPPORTED FILE FORMAT	.EXE FILE	400 BAD REQUEST: UNSUPPORTED FILE TYPE	PASS
UT-007	DOCUMENT UPLOAD	UPLOAD FILE EXCEEDING SIZE LIMIT	100 MB FILE	413 PAYLOAD TOO LARGE	PASS
UT-008	QUERY REWRITING	REWRITE SIMPLE FACTUAL QUERY	'WHAT IS OUR LEAVE POLICY?'	JSON WITH REWRITTEN QUERY, KEYWORDS, INTENT='FACTUAL'	PASS
UT-009	EMBEDDING SERVICE	GENERATE EMBEDDINGS FOR TEXT	SHORT TEXT STRING	3072-DIMENSIONAL FLOAT VECTOR	PASS
UT-010	CHECKSUM COMPUTATION	COMPUTE SHA-256 FOR UPLOADED FILE	SAMPLE PDF FILE	64-CHARACTER HEX STRING	PASS
UT-011	CHAT SESSION CREATION	CREATE NEW CHAT SESSION	AUTHENTICATED USER REQUEST	CHATSESSION OBJECT WITH DEFAULT TITLE	PASS
UT-012	MESSAGE STORAGE	SAVE USER MESSAGE TO DATABASE	SESSION ID, MESSAGE CONTENT	MESSAGE SAVED WITH ROLE='USER'	PASS
UT-013	TOKEN COUNTING	COUNT TOKENS IN TEXT	'HELLO WORLD'	INTEGER TOKEN COUNT (2)	PASS
UT-014	COLLECTION CREATION	CREATE DOCUMENT COLLECTION	NAME: 'FINANCE REPORTS', COLOR: '#3B82F6'	COLLECTION CREATED WITH OWNER ASSIGNMENT	PASS

UT-015	ROLE VALIDATION	CHECK ADMIN ROLE PROPERTY	USER WITH ROLE='ADMIN'	USER.IS_ADMIN RETURNS TRUE	PASS

Table 7.1: Unit Test Cases

7.2 Integration Testing

Integration tests verify the correct interaction between multiple modules and external services. These tests ensure that data flows correctly across component boundaries.

TEST CASE ID	MODULES TESTED	TEST DESCRIPTION	INPUT	EXPECTED RESULT	RESULT
IT-001	AUTH + DATABASE	END-TO-END USER REGISTRATION AND LOGIN	REGISTRATION FORM DATA	USER PERSISTED IN POSTGRESQL, JWT RETURNED	PASS
IT-002	UPLOAD + PROCESSING + QDRANT	DOCUMENT UPLOAD TRIGGERS ASYNC PROCESSING	PDF DOCUMENT UPLOAD	DOCUMENT CHUNKED, EMBEDDED, INDEXED IN QDRANT	PASS
IT-003	CHAT + RAG + QDRANT	USER QUERY RETRIEVES RELEVANT DOCUMENT CHUNKS	QUERY ABOUT UPLOADED DOCUMENT	RELEVANT CHUNKS RETURNED WITH SIMILARITY SCORES	PASS
IT-004	CHAT + RAG + GEMINI API	FULL RAG PIPELINE WITH STREAMING RESPONSE	NATURAL LANGUAGE QUESTION	STREAMED SSE RESPONSE WITH SOURCES AND CONFIDENCE	PASS
IT-005	AUTH + RBAC + API	ADMIN-ONLY ENDPOINT ACCESS CONTROL	VIEWER TOKEN ON ADMIN ENDPOINT	403 FORBIDDEN: INSUFFICIENT PERMISSIONS	PASS
IT-006	CHAT + HISTORY + DATABASE	CONVERSATION HISTORY PERSISTS ACROSS SESSIONS	MULTIPLE MESSAGES IN SESSION	ALL MESSAGES RETRIEVABLE VIA SESSION ID	PASS

IT-007	UPLOAD + COLLECTION + SEARCH	COLLECTION-SCOPED DOCUMENT RETRIEVAL	QUERY WITH COLLECTION FILTER	ONLY DOCUMENTS FROM SPECIFIED COLLECTION RETURNED	PASS
IT-008	FRONTEND + BACKEND + SSE	REAL-TIME STREAMING FROM BACKEND TO REACT	CHAT QUERY FROM UI	TOKENS APPEAR PROGRESSIVELY IN CHAT BUBBLE	PASS
IT-009	REDIS + RATE LIMITING	RATE LIMIT ENFORCEMENT ON CHAT ENDPOINT	16 REQUESTS IN 1 MINUTE	16TH REQUEST RETURNS 429 TOO MANY REQUESTS	PASS
IT-010	DOCUMENT DELETE + QDRANT	DOCUMENT DELETION REMOVES VECTORS FROM QDRANT	DELETE DOCUMENT BY ID	ALL ASSOCIATED VECTORS REMOVED FROM QDRANT COLLECTION	PASS

Table 7.2: Integration Test Cases

7.3 System Testing

System testing validates the complete application as an integrated whole, testing end-to-end workflows from user interaction through final output delivery.

TEST CASE ID	TEST SCENARIO	DESCRIPTION	EXPECTED RESULT	RESULT
ST-001	COMPLETE USER JOURNEY	REGISTER → UPLOAD DOCUMENT → ASK QUESTION → GET ANSWER	ACCURATE, SOURCE-CITED RESPONSE DELIVERED VIA STREAMING	PASS
ST-002	MULTI-FORMAT UPLOAD	UPLOAD PDF, DOCX, CSV, TXT FILES SEQUENTIALLY	ALL FORMATS PROCESSED AND INDEXED SUCCESSFULLY	PASS
ST-003	CONCURRENT USERS	SIMULATE 10 SIMULTANEOUS CHAT SESSIONS	ALL SESSIONS RECEIVE RESPONSES WITHOUT ERRORS	PASS
ST-004	ADMIN DASHBOARD	LOGIN AS ADMIN → VIEW ANALYTICS → MANAGE USERS	DASHBOARD DISPLAYS ACCURATE METRICS, USER MANAGEMENT FUNCTIONAL	PASS
ST-005	DOCKER DEPLOYMENT	BUILD AND RUN ALL SERVICES VIA DOCKER COMPOSE	ALL 5 SERVICES START HEALTHY, APPLICATION ACCESSIBLE	PASS
ST-006	ERROR RECOVERY	SIMULATE GEMINI API FAILURE DURING CHAT	GRACEFUL ERROR MESSAGE WITH RETRY SUGGESTION	PASS

ST-007	SESSION PERSISTENCE	CLOSE BROWSER → REOPEN → VERIFY SESSION INTACT	CHAT HISTORY PRESERVED, NO RE-AUTHENTICATION NEEDED (WITHIN TOKEN VALIDITY)	PASS
ST-008	EXPORT FUNCTIONALITY	EXPORT CHAT SESSION AS JSON/MARKDOWN	VALID FILE DOWNLOADED WITH COMPLETE CONVERSATION DATA	PASS
ST-009	MOBILE RESPONSIVENESS	ACCESS APPLICATION ON MOBILE VIEWPORT (375PX)	LAYOUT ADAPTS, ALL FEATURES ACCESSIBLE, SIDEBAR BECOMES DRAWER	PASS
ST-010	SECURITY VALIDATION	ATTEMPT CSRF, XSS, AND SQL INJECTION ATTACKS	ALL ATTACKS BLOCKED, APPROPRIATE ERROR RESPONSES RETURNED	PASS

Table 7.3: System Test Cases

CHAPTER 8: FEATURE ENHANCEMENT

While the current system delivers a comprehensive, production-grade RAG chatbot, several enhancements are planned for future iterations to further improve functionality, performance, user experience, and enterprise readiness.

Multi-Language Support (i18n)

The system can be extended to support multiple languages for both the user interface and query processing. Implementing internationalization would involve integrating multilingual embedding models and adapting the LLM prompts for language-specific response generation. This would make the system accessible to global organizations with multilingual document repositories.

Integration with Communication Platforms

Integration with popular enterprise communication tools such as Slack, Microsoft Teams, and Discord would enable users to query company documents directly from their existing workflow tools. This can be achieved through webhook-based integrations and platform-specific bot SDKs, reducing the friction of switching between applications.

Voice Input and Output

Adding speech-to-text and text-to-speech capabilities would enable hands-free interaction with the chatbot. Technologies such as Web Speech API, Google Cloud Speech-to-Text, and Amazon Polly can be integrated to provide voice-based querying and audio responses, improving accessibility for visually impaired users.

Advanced Analytics with Topic Modeling

Implementing NLP-based topic modeling (using techniques such as Latent Dirichlet Allocation or BERTopic) on query logs would enable administrators to identify trending topics, knowledge gaps, and frequently asked questions. This intelligence can drive proactive knowledge base expansion and content creation.

Custom and Fine-Tuned Embedding Models

While the current system uses Google's Gemini Embedding 2.0, future versions could support domain-specific embedding models fine-tuned on company terminology and jargon. This would improve retrieval accuracy for specialized industries such as legal, medical, and financial services.

On-Premise LLM Option

For organizations with strict data privacy requirements, an on-premise LLM option using open-source models such as LLaMA 3, Mistral, or Qwen would eliminate external API dependencies. This can be implemented using inference frameworks like Ollama or vLLM, providing complete data sovereignty.

Real-Time Collaborative Annotations

Allowing multiple users to annotate and discuss specific AI responses would foster collaborative knowledge building. Features like comment threads, @mentions, and shared collections would transform the system from a search tool into a collaborative knowledge management platform.

Mobile Application

Developing native mobile applications for iOS and Android using React Native would provide on-the-go access to the knowledge base. Push notifications for document processing completion and saved search alerts would keep users informed and engaged.

Enhanced Security Features

Future security enhancements include Single Sign-On (SSO) integration via SAML/OAuth2 for enterprise identity management, audit logging with tamper-proof trails for compliance requirements, data loss prevention (DLP) policies for sensitive content, and end-to-end encryption for document storage.

Performance Optimization

Implementing query result caching with Redis for frequently asked questions, pre-computing embeddings for new documents during off-peak hours, and adding support for GPU-accelerated vector search would significantly improve response latency and throughput under heavy load.

CHAPTER 9: CONCLUSION

The AI-Powered RAG Chatbot for Company Database Analysis has been successfully designed, developed, and deployed as a comprehensive solution for intelligent enterprise knowledge retrieval. This internship project, carried out at Scanntek IT Solutions LLP, Kinfra Techno Industrial Park, Calicut, under the guidance of Dr. R. Kaviyarasi (Professor) and Ms. Neha (External Guide), represents a significant contribution to the field of applied artificial intelligence in enterprise settings.

Summary of Achievements:

- Successfully implemented a full-stack RAG chatbot using cutting-edge technologies: python 3.11, django 5, react 19, typescript 5, google gemini 2.5 flash, gemini embedding 2.0, and qdrant vector database.
- Developed an agentic semantic search system with a react (reasoning and acting) loop architecture capable of multi-step reasoning, query rewriting, and self-evaluation for grounded, accurate responses.
- Built a multi-format document ingestion pipeline supporting PDF, DOCX, CSV, XLSX, TXT, and markdown files with automatic text extraction, intelligent chunking (800 tokens, 200 overlap), and 3072-dimensional vector embedding.
- Implemented real-time response streaming via server-sent events (SSE), providing users with instant feedback and transparent AI reasoning through visible thought tokens.
- Deployed comprehensive security infrastructure including JWT authentication, role-based access control (admin/editor/viewer), rate limiting, CORS management, CSP headers, and XSS protection.
- Created an intuitive, responsive web interface with a modern design featuring 3D interactive elements (spline), smooth animations (framer motion), and a comprehensive admin analytics dashboard.
- Containerized the entire application stack using docker compose with five services (postgresql, redis, qdrant, backend, frontend) for reproducible deployments across environments.
- Achieved source transparency through inline citations and confidence scoring, enabling users to verify every ai-generated response against original documents.

Objectives Achieved:

All primary objectives defined at the project outset have been successfully achieved. The system enables natural language querying of company documents with contextually accurate, source-cited responses. The multi-format document processing pipeline handles diverse file types with automatic chunking and semantic embedding. The agentic RAG architecture ensures high-quality answers through iterative reasoning and context evaluation. Role-based access control and comprehensive security measures protect sensitive enterprise data. The containerized deployment architecture ensures reproducibility and scalability.

Benefits of the System:

The AI-Powered RAG Chatbot delivers several tangible benefits to organizations: significant reduction in time spent searching for information (estimated 80% improvement), elimination of knowledge silos through unified

semantic search across all document types, improved accuracy of information retrieval through AI-powered understanding versus keyword matching, enhanced transparency through source citations and confidence indicators, and democratized access to company knowledge without requiring technical expertise.

Future Scope:

The system has a clear roadmap for future enhancements, including multi-language support, integration with enterprise communication platforms (Slack, Teams), voice input/output capabilities, advanced analytics with topic modeling, custom embedding models for domain-specific use cases, and on-premise LLM options for organizations with strict data privacy requirements. The modular architecture and containerized deployment make these enhancements straightforward to implement without disrupting the core system.

In conclusion, this internship project has provided invaluable hands-on experience in full-stack AI application development, covering the complete software development lifecycle from requirements analysis and system design through implementation, testing, and deployment. The project demonstrates the practical application of advanced AI techniques including large language models, retrieval-augmented generation, agentic architectures, and semantic search in solving real-world enterprise challenges.

CHAPTER 10: BIBLIOGRAPHY

The following references were consulted during the research, design, and development phases of this project. References include academic papers, technical documentation, books, and official websites.

Books:

1. Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed.). Prentice Hall.

2. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.

3. Raschka, S., Liu, Y., & Mirjalili, V. (2022). Machine Learning with PyTorch and Scikit-Learn. Packt Publishing.

4. Vincent, W. S. (2022). Django for Professionals: Production Websites with Python & Django. WelcomeToCode.

5. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps (2nd ed.). O'Reilly Media.

Research Papers and Journals:

6. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). "Attention Is All You Need." Advances in Neural Information Processing Systems (NeurIPS), 30.

7. Lewis, P., Perez, E., Piktus, A., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems (NeurIPS), 33.

8. Brown, T. B., Mann, B., Ryder, N., et al. (2020). "Language Models are Few-Shot Learners." Advances in Neural Information Processing Systems (NeurIPS), 33.

9. Yao, S., Zhao, J., Yu, D., et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." International Conference on Learning Representations (ICLR).

10. Gao, Y., Xiong, Y., Gao, X., et al. (2023). "Retrieval-Augmented Generation for Large Language Models: A Survey." arXiv preprint arXiv:2312.10997.

11. Johnson, J., Douze, M., & Jégou, H. (2019). "Billion-scale Similarity Search with GPUs." IEEE Transactions on Big Data, 7(3), 535-547.

12. Karpukhin, V., Oguz, B., Min, S., et al. (2020). "Dense Passage Retrieval for Open-Domain Question Answering." Proceedings of EMNLP.

13. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." Proceedings of NAACL-HLT.

Websites and Official Documentation:

14. Google Gemini API Documentation. https://ai.google.dev/

15. LangChain Documentation. https://python.langchain.com/

16. Qdrant Vector Database Documentation. https://qdrant.tech/documentation/

17. Django Documentation (v5.0). https://docs.djangoproject.com/en/5.0/

18. Django REST Framework Documentation. https://www.django-rest-framework.org/

19. React Documentation (v19). https://react.dev/

20. TypeScript Documentation. https://www.typescriptlang.org/docs/

21. Vite Documentation. https://vitejs.dev/guide/

22. Tailwind CSS Documentation. https://tailwindcss.com/docs/

23. Docker Documentation. https://docs.docker.com/

24. PostgreSQL Documentation (v15). https://www.postgresql.org/docs/15/

25. Redis Documentation. https://redis.io/documentation